

Done! I created an Excel workbook containing **12 worksheets (January–December 2024)**.

Each monthly sheet contains **180 sample IT helpdesk calls** with the following columns:

Column	Description
S.No	Serial number
Date	Call logged date
Problem	System, LAN, Network, Printer, PC Slow, VPN, Outlook, etc.
User Name	Randomly selected from 300 users
Phone (Extension)	Extension number
Department	One of 20 departments
Action Taken	Appropriate resolution based on the issue
Date of Completion	Completion date
Status	Completed
Time Taken (Hours)	1–4 hours
Solved By	Srinu, Gyaneshwar, Bhanu, Hameed, Prabhakar, or Mahesh

You can download it here:

 **IT_Helpdesk_Calls_2024_Monthly.xlsx**

If you're building a **Power BI dashboard**, I can also generate a much more realistic dataset with:

- **1000–5000 calls per year**
- Priority (Low, Medium, High, Critical)
- Building (7 buildings)
- 20 departments
- 300 named users

- Asset ID / Computer Name
- Call Category & Subcategory
- SLA (4h, 8h, 24h)
- Technician assignment based on workload
- Reopened calls
- Escalations
- Pending, In Progress, Completed, Cancelled
- Actual resolution times
- Holidays and weekends
- Monthly trends and seasonal spikes
- Perfect relationships for Power BI dashboards and DAX

This would look like data from a real corporate IT helpdesk rather than randomly generated records.

IT_Helpdesk_Calls_2024_Monthly.xlsx SpreadsheetOpen file

Yes. This is actually a very good real-world Power BI + Machine Learning project. You don't need a database initially. You can keep **one folder** where every month's Excel file is placed, and Power BI will automatically refresh whenever a new file is added.

Project Architecture

Users Raise Calls

|



Monthly Excel Files

(Jan.xlsx, Feb.xlsx...)

|



Shared Folder

D:\IT_Helpdesk_Calls\

|



Power BI

(Power Query → Combine Files)

|

|_____|

| |



Dashboard Machine Learning

| |

|_____Prediction & Insights_____|

Folder Structure

D:\IT_Helpdesk_Calls\

2024_January.xlsx

2024_February.xlsx

2024_March.xlsx

...

2025_January.xlsx

2025_February.xlsx

...

Every file has exactly the same columns.

Power BI automatically combines them.

Step 1

Load Folder

Instead of

Get Data

→ Excel

Use

Get Data

↓

Folder

↓

D:\IT_Helpdesk_Calls\

Power BI will show

2024_January.xlsx

2024_February.xlsx

2024_March.xlsx

...

Click

Combine Files

Power Query automatically merges every file.

Whenever a new file is copied into the folder

2026_August.xlsx

Refresh

↓

Power BI loads it automatically.

No editing required.

Data Model

Fact Table

FactCalls

Contains

Date

Problem

User

Department

Extension

Technician

Time Taken

Status

Solution

Completion Date

Dimension Tables

Date Table

Department Table

Technician Table

Problem Category

Users Table

Star Schema

Department

|

Technician – Fact Calls – Users

|

Calendar

Dashboard Pages

Executive Dashboard

KPIs

Total Calls

Completed Calls

Pending Calls

Average Resolution Time

SLA %

Reopened Calls

High Priority Calls

Cards

✓ Total Calls

✓ Completed

✓ Avg Hours

✓ Technicians

✓ Departments

Calls Analysis

Charts

Calls by Month

Calls by Week

Calls by Day

Calls by Hour

Find

Busy months

Busy days

Busy timings

Department Analysis

Top Departments

Bottom Departments

Department-wise Calls

Average Resolution Time

Example

Finance

HR

Admin

IT

CMD Office

Technician Dashboard

Every technician

Srinu

Bhanu

Mahesh

Hameed

Gyaneshwar

Prabhakar

See

Assigned Calls

Completed

Average Time

Performance

Workload

Efficiency

Leaderboard

 Top Performer

Lowest Resolution Time

Most Calls Solved

Problem Analysis

Pie Chart

Printer

Network

System

Slow PC

Email

VPN

Application

Password

Treemap

Problem Category

↓

Subcategory

SLA Dashboard

Resolved <4 Hours

Resolved 4–8 Hours

Exceeded SLA

Critical Delays

Gauge

SLA %

Target 95%

User Analysis

Top users

Who raises most tickets?

Heatmap

Department

↓

User

↓

Calls

Building Dashboard

If

7 Buildings

Then

Building-wise Calls

Building-wise Technician

Average Time

Map

or

Bar Chart

Root Cause Dashboard

Top 10 Problems

Most Frequent Printer

Most Frequent PC

Repeat Calls

DAX Measures

Total Calls

COUNTROWS(FactCalls)

Completed

Completed Calls =

CALCULATE(
COUNTROWS(FactCalls),
FactCalls[Status]="Completed")

Average Resolution

Average Hours =

AVERAGE(FactCalls[Time Taken])

SLA

SLA % =

DIVIDE(
Completed within SLA,
Total Calls)

Monthly Growth

Month-over-Month %

Previous Month Comparison

Power Query

Automatic

Folder



Combine Files



Remove Errors



Change Data Types

↓

Append Files

↓

Load

Machine Learning

Instead of just reports,
predict future calls.

Model 1

Predict Ticket Volume

Input

Month

Department

Building

Day

Holiday

Previous Month Calls

Output

Expected Calls Next Month

Algorithms

Linear Regression

Random Forest Regressor

XGBoost

Model 2

Predict Resolution Time

Target

Time Taken

Features

Problem

Technician

Department

Priority

Building

Algorithms

Decision Tree

Random Forest

Gradient Boosting

Prediction

This ticket

↓

Expected

2.8 Hours

Model 3

Predict Technician

Classification

Input

Problem

Department

Priority

Building

Output

Best Technician

Algorithms

Random Forest

CatBoost

XGBoost

Model 4

Priority Prediction

Instead of manually selecting

High

Medium

Low

ML predicts automatically.

Model 5

Repeated Failure Detection

Example

Printer-12

↓

15 tickets

↓

Abnormal

Use

Isolation Forest

DBSCAN

Anomaly Detection

Model 6

Technician Performance Score

ML predicts

Efficiency

Expected Delay

Future Workload

Python Integration

Excel Folder

↓

Python

↓

Train Model

↓

Save Model.pkl

↓

Predictions.csv

↓

Power BI



Dashboard

Python libraries

Pandas

NumPy

Scikit-learn

XGBoost

LightGBM

Joblib

Automatic Refresh

Every Month



New Excel File



Copy into Folder



Power BI Refresh



Dashboard Updated



Python Retrains Model



Predictions Updated

You can schedule this using Windows Task Scheduler or Power BI Service (with an on-premises data gateway if the files are on a local machine).

Final Dashboard

Executive Dashboard



Department Analysis



Technician Performance



Problem Trends



SLA Dashboard



Building Analysis



Prediction Dashboard



Anomaly Detection

↓

Forecast Dashboard

This end-to-end project is strong enough for a portfolio because it combines **Excel automation, Power Query, Power BI, DAX, Python, machine learning, forecasting, anomaly detection, and automated monthly refreshes** in a realistic IT helpdesk scenario.

If you want to **exclude Machine Learning** and build a **professional Power BI Dashboard** like those used in IT organizations, here's a complete end-to-end design.

Project Flow

Users Raise Complaints

|



Monthly Excel Files

(January.xlsx, February.xlsx...)

|



Shared Folder

(D:\IT_Helpdesk_Calls)

|



Power Query

(Combine Files)

|



Data Cleaning & Transformation

|



Data Model

(Star Schema)

|



DAX Measures

|



Interactive Power BI Dashboard

Folder Structure

D:\IT_Helpdesk_Calls

2024_January.xlsx

2024_February.xlsx

2024_March.xlsx

...

2025_January.xlsx

...

Every file should have **exactly the same columns**.

When you add a new month's file, simply click **Refresh** in Power BI (or schedule a refresh in the Power BI Service).

Power Query Steps

Get Data

Get Data



Folder



Select IT_Helpdesk_Calls Folder

Power BI automatically detects every Excel file.

Click

Combine Files

Power Query will append all months into one table.

Data Cleaning

- Remove blank rows
- Remove duplicates
- Convert Date columns to Date type
- Convert Time Taken to Whole Number
- Rename columns
- Trim spaces
- Standardize department names

- Standardize technician names
-

Data Model

Fact Table

FactCalls

Contains

- Date
 - User
 - Department
 - Problem
 - Technician
 - Resolution Time
 - Status
 - Completion Date
-

Dimension Tables

Calendar

- Date
- Month
- Quarter
- Year
- Week

Department

20 departments

Technician

- Srinu
- Gyaneshwar
- Bhanu
- Hameed
- Prabhakar
- Mahesh

Problem Category

- Printer
 - Network
 - System
 - Slow PC
 - Email
 - VPN
 - Others
-

Dashboard Pages

Page 1 — Executive Dashboard

KPI Cards

- Total Calls
- Completed Calls
- Pending Calls
- Average Resolution Time

- Number of Departments
 - Number of Technicians
-

Charts

Calls by Month

Visual

Clustered Column Chart

Shows

January

February

March

...

Calls by Department

Visual

Bar Chart

Shows

HR

Finance

IT

Admin

Stores

Calls by Category

Visual

Donut Chart

Displays

- Printer
 - Network
 - System
 - Email
 - Slow PC
-

Calls by Status

Visual

Pie Chart

Completed

Pending

Cancelled

Page 2 — Department Analysis

Visuals

Department-wise Calls

Bar Chart

Department-wise Average Resolution Time

Column Chart

Top 10 Departments

Horizontal Bar Chart

Department Trend

Line Chart

Month

vs

Number of Calls

Page 3 — Technician Dashboard

Technician Performance

Cards

- Total Assigned

- Completed
 - Average Resolution Time
-

Calls Solved

Bar Chart

Srinu

Bhanu

Mahesh

Hameed

Resolution Time

Column Chart

Technician

vs

Average Hours

Technician Monthly Trend

Line Chart

Page 4 — Problem Analysis

Problem Distribution

Donut Chart

Printer

Network

System

VPN

Email

Problem Trend

Line Chart

Month

vs

Problem Count

Top 10 Problems

Bar Chart

Resolution Time by Problem

Column Chart

Page 5 — Time Analysis

Calls by Month

Line Chart

Calls by Day

Column Chart

Calls by Week

Line Chart

Calls by Hour *(if available)*

Area Chart

Page 6 — User Analysis

Top Users

Bar Chart

Users generating maximum tickets

Calls by Department and User

Matrix

Department

↓

Users

↓

Calls

Page 7 — SLA Dashboard

KPI

Average Resolution Time

Calls within SLA

Gauge

Target

95%

Calls Exceeding SLA

Bar Chart

Resolution Time Distribution

Histogram

Page 8 — Completion Dashboard

Completed

Pending

Cancelled

Stacked Column Chart

Month-wise

Page 9 — Trend Dashboard

Monthly Calls

Line Chart

Monthly Growth

Area Chart

Running Total

Line Chart

Year-over-Year Comparison (*when multiple years are available*)

Clustered Column Chart

Slicers (Filters)

Add slicers on every page for:

- Year
- Month
- Department

- Technician
 - Problem Category
 - Status
 - User Name
-

DAX Measures

Total Calls

DAX

Total Calls =

COUNTROWS(FactCalls)

Completed Calls

DAX

Completed Calls =

CALCULATE(
COUNTROWS(FactCalls),
FactCalls[Status] = "Completed"
)

Pending Calls

DAX

Pending Calls =

CALCULATE(
COUNTROWS(FactCalls),

FactCalls[Status] = "Pending"

)

Average Resolution Time

DAX

Average Resolution Time =

AVERAGE(FactCalls[Time Taken])

Completion Rate

DAX

Completion % =

DIVIDE(

[Completed Calls],

[Total Calls]

)

Format as Percentage.

Calls This Month

DAX

Calls This Month =

TOTALMTD(

[Total Calls],

Calendar[Date]

)

Calls This Year

DAX

Calls This Year =

TOTALYTD(

[Total Calls],

Calendar[Date]

)

Recommended Theme

- Background: White or light gray
 - Primary color: Blue
 - Accent color: Green for completed, Orange for pending, Red for overdue
 - Use consistent fonts and spacing
-

Automatic Refresh

1. Store all monthly Excel files in the same folder.
 2. Use **Get Data** → **Folder** and **Combine Files** in Power Query.
 3. Save the Power BI report.
 4. When a new month's Excel file is added to the folder, click **Refresh** (or configure a scheduled refresh in the Power BI Service if using OneDrive/SharePoint or an on-premises gateway).
-

Final Dashboard Structure

1. Executive Overview

2. Department Analysis
3. Technician Performance
4. Problem Analysis
5. Time Analysis
6. User Analysis
7. SLA Dashboard
8. Completion Status
9. Monthly Trends

This layout closely matches the style of dashboards used by corporate IT helpdesk teams, providing management with a clear view of ticket volume, service performance, workload distribution, and operational trends without requiring machine learning.

Absolutely. In fact, this is one of the best beginner-to-advanced Power BI projects because it teaches almost every important concept: **Power Query, data modeling, DAX, visualization, filters, drill-through, bookmarks, and automatic refresh.**

A detailed, end-to-end guide would be extremely long (well over 100 pages with screenshots if written completely in chat), so the most effective approach is to build it as a structured course.

Course Outline

Phase 1: Preparing the Data

- Install Power BI Desktop
- Organize your Excel files
- Understand the dataset
- Create the folder structure
- Verify that every month's Excel file has the same columns

Phase 2: Importing Data

- Get Data → Folder

- Why we use Folder instead of Excel
- Combine Files
- How Power Query creates the sample query
- Understanding all automatically created queries
- Loading the combined table

Phase 3: Power Query

- Remove unnecessary columns
- Rename columns
- Change data types
- Replace values
- Split columns
- Merge columns
- Create custom columns
- Add Month Name
- Add Month Number
- Add Year
- Add Quarter
- Add Week Number
- Add Day Name
- Handle errors
- Remove duplicates
- Apply and Close

Phase 4: Data Modeling

- Understand Star Schema
- Create Calendar Table

- Create Department Table
- Create Technician Table
- Create Problem Category Table
- Relationships
- One-to-many relationships
- Filter direction
- Hide unnecessary columns

Phase 5: DAX

Build measures such as:

- Total Calls
- Completed Calls
- Pending Calls
- Average Resolution Time
- Completion %
- Calls This Month
- Calls This Year
- Previous Month Calls
- Monthly Growth %
- Running Total
- Rank Departments
- Rank Technicians

Phase 6: Dashboard Design

Create pages including:

1. Executive Dashboard

2. Department Dashboard
3. Technician Dashboard
4. Problem Dashboard
5. Time Analysis
6. User Analysis
7. SLA Dashboard
8. Trends Dashboard

Phase 7: Interactive Features

- Slicers
- Drill-through
- Tooltips
- Sync slicers
- Bookmarks
- Buttons
- Navigation
- Conditional formatting
- Dynamic titles

Phase 8: Publishing

- Publish to Power BI Service
 - Configure scheduled refresh
 - Refresh automatically when a new monthly Excel file is added
 - Share the report
-

How I Recommend We Build It

Instead of giving a huge wall of text, we build the project exactly like a live classroom.

Each lesson will include:

- Objective
- Why the step is needed
- What Power BI is doing internally
- Common mistakes
- Best practices
- Hands-on exercise
- Expected output before moving on

For example:

Lesson 1 – Creating the Folder

We'll cover:

- Why Power BI works better with a folder than a single Excel file
- How to organize monthly files
- Naming conventions
- Verifying identical schemas
- What happens when a file has an extra or missing column

Lesson 2 – Importing the Folder

We'll cover:

- Get Data → Folder
- What "Combine Files" actually does
- Understanding the automatically generated helper queries
- Inspecting the Power Query steps

- Why future files are picked up automatically

And so on until the complete dashboard is finished.

What You'll Build

By the end, you'll have a professional IT Helpdesk dashboard with:

- Executive KPIs
- Monthly call trends
- Department-wise analysis
- Technician performance
- Problem category breakdown
- Resolution time analysis
- Interactive slicers and drill-through
- Automatic monthly data refresh from the folder

This will closely resemble dashboards used by enterprise IT support teams and is an excellent portfolio project.

Given your interest in mastering topics from first principles, I'll explain not just **how** to perform each action in Power BI, but also **why** it works, what happens behind the scenes, and how to troubleshoot common issues at every stage.

Excellent. We'll build this as if you're working in an actual company. **Don't skip any step**, because every later step depends on the previous one.

STEP 1: Prepare the Project Structure

Goal

Before opening Power BI, organize your data properly so that **Power BI can automatically refresh every month** without changing the report.

This is one of the most important steps in a real-world project.

Why is this step important?

Imagine this situation.

Today you have:

January.xlsx

February.xlsx

March.xlsx

After one month, the IT team sends:

April.xlsx

After another month:

May.xlsx

If you import each Excel file individually into Power BI, you'll have to modify the report every month.

That is **not how companies work**.

Instead, Power BI should automatically detect every new file placed in a folder.

Real Company Scenario

Suppose your organization receives one Excel file every month.

IT Department

|

|



Monthly Call Report

|



Shared Folder

Power BI monitors that folder.

Whenever a new file appears:

2024_August.xlsx

You simply click **Refresh** (or let Power BI Service refresh on a schedule).

The dashboard updates automatically.

Create the Folder

Create a folder anywhere.

Example

D:\PowerBI Projects\

Inside it

IT Helpdesk Dashboard

Inside it

Data

Final structure

D:\PowerBI Projects\

|



IT Helpdesk Dashboard

|



Data

Why use a separate Data folder?

Because later you will also have

Images

Power BI File

Documentation

Screenshots

Reports

Keeping everything together makes the project easy to manage.

Example

IT Helpdesk Dashboard

|

|— Data

|— Reports

|— Screenshots

|— Documentation

|— Helpdesk.pbix

This is a common project structure in many organizations.

Put Excel Files into the Folder

Example

Data

|

|— 2024_January.xlsx

|— 2024_February.xlsx

|— 2024_March.xlsx

|— 2024_April.xlsx

|— 2024_May.xlsx

All monthly files go into the same folder.

Why must every Excel file have the same columns?

Suppose January contains

Date	User	Problem	Department
------	------	---------	------------

but February contains

Date	User	Problem	Building	Department
------	------	---------	----------	------------

Now Power BI doesn't know how to combine them cleanly because one file has an extra column.

It can still import the data, but you'll get **null values** for the missing column in the other months, and your report may need additional handling.

A consistent schema avoids these issues.

Correct Example

Every monthly file should have exactly the same headers.

S.No	Date	Problem	User Name	Phone	Department	Action Taken	Completion Date	Status	Time Taken	Solved By
------	------	---------	-----------	-------	------------	--------------	-----------------	--------	------------	-----------

Notice:

- Same column names
 - Same order
 - Same data types
-

Wrong Example

January

| Date | User | Problem |

February

| User | Date | Problem | Building |

This inconsistency can make maintenance harder and may require extra transformation steps.

Naming Convention

Instead of

jan.xlsx

feb.xlsx

march_new.xlsx

latest.xlsx

Use

2024_January.xlsx

2024_February.xlsx

2024_March.xlsx

Benefits:

- Files sort correctly.
 - Easy to identify the month.
 - Easier for everyone on the team to understand.
-

What happens behind the scenes?

When you choose **Get Data → Folder**, Power BI scans the folder.

Internally, it builds a table similar to this:

File Name	Folder Path	Extension	Date Modified
January.xlsx	D:\Data	.xlsx	...
February.xlsx	D:\Data	.xlsx	...
March.xlsx	D:\Data	.xlsx	...

Power Query then opens each file, extracts the relevant worksheet or table, and combines the rows into a single dataset.

Conceptually:

January

|

February

|

March

|

April

|



One Combined Table

You don't have to append files manually.

Best Practices

- Use one folder only for the monthly data files.
 - Keep the same worksheet name (optional but recommended).
 - Keep the same headers in every file.
 - Store dates as actual Excel dates, not text.
 - Avoid merged cells, blank header rows, or decorative titles above the data.
 - Format the data as an **Excel Table** (Ctrl + T) if possible. This makes the files more robust and easier to maintain.
-

Common Mistakes

✗ Saving files with different column names.

✗ Adding extra title rows like:

IT Helpdesk Report

January 2024

above the headers.

✗ Mixing text and numbers in the same column.

✗ Leaving blank rows inside the dataset.

✗ Renaming columns in only one month's file.

End Result of Step 1

Your project folder should look like this:

IT Helpdesk Dashboard

|

|— Data

| |— 2024_January.xlsx

| |— 2024_February.xlsx

| |— 2024_March.xlsx

| |— ...

| |— 2024_December.xlsx

|

|— Reports

|— Documentation

└─ Screenshots

└─ Helpdesk.pbix (this will be created later)

Before moving to Step 2, verify this checklist:

- ☒ All monthly files are in the same Data folder.
- ☒ Every file has the same worksheet structure and column headers.
- ☒ Dates are stored as dates, not text.
- ☒ No blank rows or merged header cells.
- ☒ Files follow a consistent naming convention.

Once this foundation is in place, **Step 2** will show you how Power BI reads the entire folder, automatically combines all monthly files, and prepares them for analysis.

For your project, **both approaches work**, but one is much better depending on your goal.

Option 1: One Excel File with 12 Worksheets

Example:

IT_Helpdesk_Calls.xlsx

└─ January

└─ February

└─ March

└─ April

└─ May

└─ June

└─ July

└─ August
└─ September
└─ October
└─ November
└─ December

Advantages

- Only one file to manage.
- Easy to email.
- Good for small datasets.

Disadvantages

- Every month you must **open the same file** and add a new worksheet or append data.
 - If two people edit it simultaneously, conflicts can occur.
 - If the file gets corrupted, all months are affected.
 - In Power BI, combining multiple worksheets is possible but is less scalable than reading a folder of files.
-

Option 2: One Excel File per Month (Recommended)

Example:

Data

└─ 2024_January.xlsx
└─ 2024_February.xlsx
└─ 2024_March.xlsx

└─ ...
└─ 2025_January.xlsx
└─ 2025_February.xlsx

Advantages

- This is how many organizations archive monthly reports.
- You simply drop the new month's file into the folder.
- Power BI automatically includes it after refresh.
- Older files remain unchanged.
- Easier to back up and share.
- Works perfectly with **Get Data → Folder**.

Disadvantages

- More files to manage (though Power BI handles this easily).
-

Option 3: One Excel File with a Single Sheet (Best for Growing Data)

Example:

IT_Helpdesk_Calls.xlsx

Worksheet:

Calls

The worksheet contains **all records**:

Date	Month	Year	User	Problem	Department	Status
01-Jan-2024	January	2024	User1	Printer	HR	Complete

Date	Month	Year	User	Problem	Department	Status
4						ted
02-Jan-2024	January	2024	User2	LAN	Finance	Completed
...	...	...	...	...	...	...
01-Feb-2024	February	2024	User10	PC Slow	Admin	Completed

Every month, you **append new rows** to the same table.

Advantages

- Simplest data model.
- Power BI imports one table directly.
- No need to combine worksheets or files.
- Very easy to build reports.

Disadvantages

- The file becomes larger over time.
 - If many people edit it, version control can become difficult.
-

Which Should You Choose?

For your project:

- **If this is a learning project: Option 2 (one file per month)** is the best choice because it teaches folder-based imports and automatic monthly refresh, which is a valuable Power BI skill.
- **If your organization actually maintains a single master Excel file where new calls are continuously added: Option 3 (one sheet with one table)** is the simplest and most efficient.

- **Option 1 (multiple worksheets in one workbook)** is the least preferred for long-term Power BI reporting because it requires extra transformation and doesn't scale as cleanly.

My recommendation

Since your goal is to build a professional IT Helpdesk Power BI dashboard with automatic monthly updates, I recommend **Option 2: one Excel file per month stored in a single folder**. It is scalable, easy to maintain, and aligns well with Power BI's **Folder** data source feature.

Yes. Since you want to **master Power BI** and build this as a **real company project**, I recommend treating it as a complete project rather than a simple dashboard.

The full guide is too large for a single response (it would exceed the response length limit), so we'll build it **module by module**, with each module explained in detail before moving to the next. By the end, you'll have a production-style dashboard.

IT Helpdesk Dashboard Project Roadmap

Module 1 – Project Planning (2 Lessons)

Lesson 1

- Understand the business requirement
- Identify stakeholders
- Decide KPIs
- Understand the dataset
- Decide project structure

Lesson 2

- Prepare Excel files
- Create folder structure
- Best practices

- Naming conventions
- Common mistakes

Output: Clean project folder ready for Power BI.

Module 2 – Power BI Installation & Interface (4 Lessons)

Lesson 3

- Install Power BI Desktop
- Home screen
- Power BI architecture
- Import vs DirectQuery
- Why Import mode for this project

Lesson 4

- Power BI interface
- Ribbon
- Report View
- Data View
- Model View
- Fields pane
- Filters pane
- Visualizations pane
- Format pane

Lesson 5

- PBIX file

- PBIT template
- Project organization

Lesson 6

- Power BI workflow
- How Power BI processes data behind the scenes

Output: Comfortable navigating Power BI.

Module 3 – Importing Data (5 Lessons)

Lesson 7

- Get Data → Folder
- Why Folder instead of Excel
- Selecting the data folder

Lesson 8

- What "Combine Files" does
- Sample File query
- Transform Sample File
- Automatically generated queries

Lesson 9

- Loading data
- Preview
- Query dependencies

Lesson 10

- Refresh process
- How new monthly files are detected

Lesson 11

- Common import errors
- Fixing schema mismatches

Output: One combined table from all monthly Excel files.

Module 4 – Power Query (12 Lessons)

You'll learn:

- Power Query interface
- Applied Steps
- Data types
- Rename columns
- Remove duplicates
- Replace values
- Split columns
- Merge columns
- Add custom columns
- Extract Month
- Extract Year
- Extract Quarter
- Extract Week
- Add conditional columns
- Append queries
- Merge queries
- Remove errors

- Close & Apply

Output: Clean dataset.

Module 5 – Data Modeling (8 Lessons)

Topics:

- What is a Star Schema?
- Fact vs Dimension tables
- Creating a Calendar table
- Department table
- Technician table
- Relationships
- Cross-filter direction
- Hide unnecessary columns

Output: Proper data model.

Module 6 – DAX Fundamentals (15 Lessons)

Create measures like:

- Total Calls
- Completed Calls
- Pending Calls
- Average Resolution Time
- Completion %
- Calls by Month

- Calls This Year
- Previous Month
- Running Total
- Rank Departments
- Rank Technicians
- Year-to-Date
- Month-to-Date
- Quarter-to-Date
- Dynamic Titles

Output: Reusable business metrics.

Module 7 – Dashboard Design (20 Lessons)

We'll build these pages one by one:

Executive Dashboard

- KPI cards
- Monthly trend
- Status
- Top departments

Department Dashboard

- Department comparison
- Drill-through

Technician Dashboard

- Technician performance
- Workload

Problem Dashboard

- Category analysis
- Resolution time

Time Dashboard

- Monthly trend
- Weekly trend
- Daily trend

User Dashboard

- Top users
- Call frequency

SLA Dashboard

- Completion rate
- SLA compliance

Trend Dashboard

- Growth analysis
 - Running totals
-

Module 8 – Interactive Reports (10 Lessons)

Learn:

- Slicers
- Sync slicers
- Bookmarks
- Buttons
- Navigation

- Drill-down
 - Drill-through
 - Tooltips
 - Conditional formatting
 - Dynamic text
-

Module 9 – Automatic Refresh (6 Lessons)

Topics:

- Folder refresh
 - New monthly file
 - Scheduled refresh
 - Power BI Service
 - Gateway (if needed)
 - Publishing reports
-

Module 10 – Professional Finishing (8 Lessons)

Topics:

- Theme
- Colors
- Icons
- Layout
- Performance optimization
- Best practices

- Documentation
 - Sharing
-

Final Project

By the end, you'll have a dashboard with:

- Executive Summary
 - Department Analysis
 - Technician Performance
 - Problem Analysis
 - Time Analysis
 - SLA Dashboard
 - User Analysis
 - Monthly Trends
 - Automatic monthly updates
-

How We'll Learn

For **every lesson**, I'll explain:

1. **What** we're going to do.
2. **Why** it's needed in a real business scenario.
3. **How** to do it in Power BI (every click and option).
4. **What Power BI is doing internally.**
5. **Common mistakes** and how to avoid them.
6. **Best practices** used in industry.
7. **Expected output** before moving to the next lesson.

This way, you'll understand not just the steps, but also the reasoning behind them.

Next lesson: Module 2, Lesson 3 – Installing Power BI Desktop and understanding the Power BI interface. We'll start by learning the workspace before importing any data.

Perfect. From now on, we'll build this **like a live classroom**, where you perform each step before moving to the next one.

Module 2 – Power BI Installation & Interface

Lesson 3 – Installing Power BI Desktop & Understanding the Workspace

Objective

By the end of this lesson, you will:

- Install Power BI Desktop.
 - Understand what Power BI Desktop actually is.
 - Learn the Power BI workflow.
 - Become familiar with the interface.
 - Know why each component exists.
-

What is Power BI?

Before installing it, understand **what Power BI does**.

Imagine you have 12 Excel files:

January.xlsx

February.xlsx

March.xlsx

...

December.xlsx

Each file contains:

Date	Problem	User	Department	Technician
------	---------	------	------------	------------

Your manager asks:

"Show me:

- Total calls
- Department-wise calls
- Technician performance
- Monthly trend
- Top recurring problems"

Doing this manually in Excel every month is slow.

Power BI automates this process.

Power BI Workflow

Think of Power BI as a pipeline:

Excel Files

|



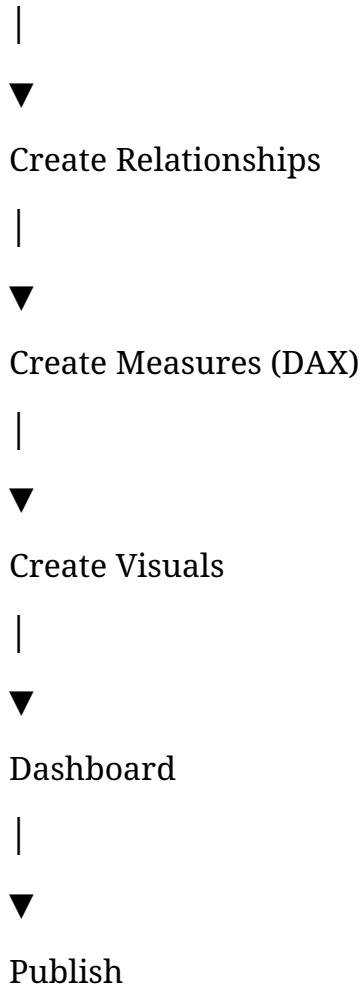
Import Data

|



Clean Data

(Power Query)



Each box represents a stage of the project.

Step 1 – Install Power BI Desktop

Go to the official Microsoft download page and install **Power BI Desktop**.

After installation, open it.

You'll see a start window with options such as:

- Get Data
- Open Report
- Recent Files

You can either close the start screen or select **Blank Report**.

Step 2 – Create Your Project Folder

Create the following structure:

IT Helpdesk Dashboard

```
|  
|— Data  
|  
|— Reports  
|  
|— Documentation  
|  
|— Images  
|  
|— Helpdesk.pbix
```

Why?

In companies:

- **Data** stores Excel files.
- **Reports** stores exported PDFs.
- **Documentation** contains project notes.
- **Images** stores logos/icons.
- **PBIX** is the Power BI project.

Keeping everything organized makes collaboration and maintenance much easier.

Step 3 – Save the PBIX File

Immediately save the new report.

Choose:

File



Save As



Helpdesk.pbix

Save it inside:

IT Helpdesk Dashboard

Why save now?

Power BI creates auto-recovery information and remembers paths to your data sources more reliably once the report has been saved.

Step 4 – Understand the Power BI Interface

When Power BI opens, you'll see several areas.

Ribbon

Report | Data | Model

Canvas

Filters | Visualizations | Data (Fields)

Let's understand each one.

1. Ribbon

Located at the top.

It contains commands such as:

- Home
- Insert
- Modeling
- View
- Optimize

Think of it like the ribbon in Microsoft Excel.

Home Tab

You'll use this the most.

Important commands:

- Get Data
- Transform Data
- Refresh
- Publish
- New Visual
- Manage Relationships

For this project, the most frequently used commands will be:

- Get Data
 - Transform Data
 - Refresh
-

2. Left Navigation

Power BI has three main views.

Report View

Icon:



Purpose:

Design dashboards.

This is where you'll spend most of your time creating charts and KPIs.

Data View

Icon:



Purpose:

View the imported data as tables.

Example:

Date	User	Problem
------	------	---------

This helps verify that the imported data is correct.

Model View

Icon:



Purpose:

Display relationships between tables.

Example:

Calendar

|

|

FactCalls

|

Department

This view becomes important after we import and model the data.

3. Report Canvas

This is the large white area.

Every visual you create—cards, charts, slicers, tables—appears here.

Think of it as a blank PowerPoint slide where you build your dashboard.

4. Filters Pane

Located on the right.

It allows you to filter:

- the entire report,
- a specific page,
- or an individual visual.

For example:

If you select **Year = 2024**, every visual on the page can update automatically (depending on the filter scope).

5. Visualizations Pane

This is where you choose chart types.

Examples:

- Bar Chart
- Column Chart
- Line Chart
- Pie Chart
- Donut Chart

- Table
- Matrix
- Card
- Gauge
- Map

You'll also use this pane to assign fields such as:

- Axis
 - Values
 - Legend
 - Tooltips
-

6. Data (Fields) Pane

This pane lists your imported tables and columns.

Example:

FactCalls

Date

Department

Problem

Technician

Status

You drag these fields into visuals to build reports.

Step 5 – Understand the Workflow (No Data Yet)

At this point, don't import anything.

Just understand the sequence:

Excel Files

|



Power Query

|



Data Model

|



DAX Measures

|



Charts

|



Dashboard

Many beginners jump straight into creating charts. In professional projects, it's better to follow this order because clean data and a good model make reporting much easier.

Common Beginner Mistakes

- ✗ Creating charts before checking the data.
 - ✗ Saving the PBIX file in a different location from the project folder.
 - ✗ Importing files before planning the project structure.
 - ✗ Ignoring the Data and Model views.
-

Lesson 3 Checklist

Before moving to the next lesson, verify that:

- ☒ Power BI Desktop is installed.
 - ☒ Your project folder structure is created.
 - ☒ Helpdesk.pbix is saved in the project folder.
 - ☒ You can identify the Report, Data, and Model views.
 - ☒ You know the purpose of the Ribbon, Canvas, Filters, Visualizations, and Data panes.
-

Next Lesson (Lesson 4)

We'll start working with **real data** by connecting Power BI to your **Data** folder. You'll learn:

- Why **Get Data → Folder** is better than **Get Data → Excel**.
- How Power BI discovers every monthly file.

- What happens internally when you click **Combine Files**.
- The helper queries Power BI creates automatically.
- Why future monthly files are included automatically after a refresh.

We'll take it step by step and inspect every screen and option so you understand both **how** and **why** Power BI behaves the way it does.

Excellent. This is where the actual project begins.

Most tutorials simply say **Get Data → Folder → Combine**.

We are **not** going to do that.

Instead, I'll explain **every button, why it exists, what Power BI does internally, and what happens after you click it**.

Module 3 — Importing Data

Lesson 4 — Import Monthly Excel Files Using Folder

Objective

At the end of this lesson, you will know:

- Why companies use **Folder** instead of **Excel**
 - How Power BI imports multiple files
 - What every option on the import screen does
 - What happens internally when importing
 - Common mistakes
-

Before Opening Power BI

Suppose your folder looks like this:

IT Helpdesk Dashboard

```
|
|
|— Data
|
|
| |
| |— 2024_January.xlsx
| |— 2024_February.xlsx
| |— 2024_March.xlsx
| |— 2024_April.xlsx
| |— ...
|
|
|— Helpdesk.pbix
```

Every Excel file contains the same columns.

Example:

S.N o	Dat e	Pro ble m	Use r Na me	Pho ne	Dep art me nt	Acti on Tak en	Co mpl etio n Dat e	Stat us	Tim e Tak en	Solv ed By
----------	----------	-----------------	----------------------	-----------	------------------------	-------------------------	------------------------------------	------------	-----------------------	------------------

Step 1

Open Power BI Desktop

Open

Helpdesk.pbix

If it is a new file,
choose
Blank Report
You should see an empty report canvas.

Step 2

Click
Home
Look at the ribbon.
You will see
Get Data
It looks like
Home

Clipboard

↓

Get Data
Click the small arrow beside **Get Data**.
Why the arrow?
Because Power BI supports many data sources.
Examples

- Excel
- SQL Server
- Oracle
- Folder
- SharePoint
- CSV
- PDF
- Web
- JSON

Power BI can connect to almost anything.

Step 3

Choose

More...

Why?

Because it shows every available connector.

A window opens.

Get Data

All

File

Database

Power Platform

Azure

Online Services

Other

Question

Why don't we click

Excel Workbook

?

Because we don't want

January.xlsx

only.

We want

January

February

March

April

May

...

December

and every future month.

Therefore

choose

Folder

Step 4

Select

Folder

Click

Connect

Now Power BI asks

Browse

Navigate to

IT Helpdesk Dashboard

↓

Data

Select the folder.

NOT

January.xlsx

NOT

February.xlsx

ONLY

Data

Click

OK

What happens internally?

Many beginners think

"Power BI is importing Excel."

Actually

No.

It first imports the **folder information**.

Internally Power BI creates something similar to

Name	Extension	Folder Path	Date Modified
January.xlsx	.xlsx	D:\Data	...
February.xlsx	.xlsx	D:\Data	...
March.xlsx	.xlsx	D:\Data	...

Notice

It has **NOT read the Excel data yet**.

It only knows

"These files exist."

Think of it like Windows File Explorer.

Power BI is reading

Folder Contents

before reading

Excel Contents

Step 5

Now you'll see something like

Content	Name	Extension	Date Created
---------	------	-----------	--------------

The important column is

Content

This contains the binary file.

Question

What is Binary?

Imagine

January.xlsx

Power BI cannot understand Excel immediately.

It first stores the file as

Binary

Think of Binary as

Raw File

Later

Power BI opens it.

Step 6

At the bottom you will see

Combine

Load

Transform Data

Cancel

Let's understand each one.

Load

If you click

Load

Power BI loads only the folder metadata.

Meaning

File Names

Extension

Folder Path

Modified Date

NOT

the Excel data.

Almost never used for reporting.

Transform Data

This opens

Power Query

WITHOUT combining the files.

Useful if

- you want only certain files

- remove temporary files
- remove old years

Example

Delete

2022

2023

Keep only

2024

2025

Professionals often choose **Transform Data** first because it gives more control.

Combine

This is the option we want.

Clicking **Combine** tells Power BI:

"Open every file in this folder, extract the data, and merge it into one table."

Step 7

Click

Combine



Combine & Transform Data

You may also see **Combine & Load**.

Which should you choose?

For this project, choose:

Combine & Transform Data

Why?

Because we still need to:

- clean data
 - change data types
 - rename columns
 - remove errors
 - prepare the dataset
-

Step 8

Power BI now asks which object to import.

Example

January.xlsx



Sheet1

Calls

Table1

If your Excel file contains an Excel Table named Calls, select that. If it contains only a worksheet, select the worksheet.

Why does Power BI ask this?

An Excel file can contain multiple objects:

Workbook

↓

January Sheet

February Sheet

Hidden Sheet

Pivot Table

Excel Table

Power BI needs to know which one contains the data you want.

Best Practice

Instead of leaving your data as plain cells,
convert it into an **Excel Table**.

In Excel:

1. Select the data.
2. Press **Ctrl + T**.
3. Check **My table has headers**.
4. Give it a meaningful name like:

Calls

Now every monthly workbook has the same table name.

This makes Power BI imports more reliable.

Step 9

Click

OK

Power BI opens **Power Query Editor**.

Don't worry about the new queries yet.

We'll explain every one in the next lesson.

What just happened behind the scenes?

When you clicked **Combine & Transform Data**, Power BI automatically created several helper queries. Conceptually, the process looks like this:

Folder

|



List Files

|



Open Each Excel File

|



Read the Selected Sheet/Table

|



Apply the Same Transformation to Every File

|



Append All Rows

|



One Combined Table

This is the key reason new monthly files can be included automatically: the same transformation steps are applied to every file in the folder.

Common Mistakes

- ✗ Choosing **Excel Workbook** instead of **Folder**.
 - ✗ Selecting the wrong worksheet or table inside the workbook.
 - ✗ Using different sheet or table names in different monthly files.
 - ✗ Having different column headers between months.
 - ✗ Clicking **Load** instead of **Combine & Transform Data** when you know you'll need to clean the data.
-

Lesson 4 Checklist

Before moving on, confirm that:

- ☒ You connected to the **Data** folder, not an individual Excel file.
 - ☒ You selected **Combine & Transform Data**.
 - ☒ You selected the correct worksheet or Excel Table.
 - ☒ Power Query Editor has opened.
-

Next Lesson (Lesson 5)

We'll stay inside **Power Query Editor** and explain **every query Power BI created automatically**, including:

- Transform Sample File
- Sample File
- Parameter1
- Helper Queries
- The main combined query

You'll learn why these queries exist, how they work together, and why you should avoid deleting them even though they may seem unnecessary.

Excellent. This is the lesson where many beginners get confused. When you click **Combine & Transform Data**, Power BI suddenly creates **4–6 queries automatically**. Many people delete them because they don't understand their purpose, which often breaks the refresh process.

In this lesson, we'll understand **every query** and what Power BI is doing internally.

Module 3 – Importing Data

Lesson 5 – Understanding Power Query and Automatically Generated Queries

Objective

By the end of this lesson, you will understand:

- What Power Query is
 - Why Power BI creates helper queries
 - What each automatically created query does
 - How they work together
 - Which queries you should never delete
-

Step 1 – What is Power Query?

When you click:

Combine & Transform Data

Power BI opens **Power Query Editor**.

Power Query is **not** Power BI's reporting area.

Think of Power BI as having two major parts:

Excel Files

|



Power Query

(Data Preparation)

|



Power BI Data Model

|



Visualizations

Power Query is responsible for:

- Reading data
- Cleaning data
- Transforming data
- Combining files
- Preparing data before it enters the data model

It does **not** create charts.

Step 2 – Understand the Power Query Window

When Power Query opens, you'll see something like:

Ribbon

Queries Pane Data Preview Query Settings
(Left) (Center) (Right)

Status Bar

Let's understand each section.

Queries Pane (Left)

This lists every query in your project.

After combining files from a folder, you may see something like:

Queries

Folder Files

Transform Sample File

Sample File

Parameter1

Transform File

Depending on your Power BI version, the names may differ slightly, but the purpose is the same.

Data Preview (Center)

This displays the current output of the selected query.

For example:

Date	Problem	Department
01-Jan	Printer	HR
02-Jan	LAN	Finance

This is a preview only.

Power BI hasn't loaded it into the report yet.

Query Settings (Right)

Contains two important sections:

Properties

Query Name

Example:

Folder Files

Applied Steps

Example

Source

Filtered Hidden Files

Invoke Custom Function

Expanded Table

Changed Type

These are the transformation steps Power Query recorded.

Think of them like a recipe.

Every time you refresh, Power BI repeats these same steps automatically.

Step 3 – What Queries Were Created?

Let's assume your folder contains:

2024_January.xlsx

2024_February.xlsx

2024_March.xlsx

Power BI creates several helper queries.

Query 1

Folder Files (Main Query)

This is your primary query.

It contains the combined data from all monthly files.

Conceptually:

January

|

February

|

March

|



Folder Files

This is the query you'll eventually load into Power BI.

Query 2

Sample File

What is it?

Power BI chooses **one** Excel file from the folder.

For example:

January.xlsx

It uses this file as an example to understand the file structure.

Think of it like this:

Power BI:

"Let me inspect one file first."



January.xlsx



"What sheet does it have?"

"What columns?"

"What data types?"

It does **not** mean only January will be imported.

It is simply a representative sample.

Why is a Sample File Needed?

Suppose your files contain:

January.xlsx



Sheet: Calls

↓

Columns:

Date

Problem

Department

Power BI learns:

"Every file should have this structure."

Then it applies the same logic to every other file.

Query 3

Transform Sample File

This is one of the most important queries.

Many beginners delete it.

Don't.

Its purpose is to define **how one file should be transformed**.

Imagine the transformation steps are:

Remove Blank Rows



Promote Headers



Change Data Types



Rename Columns

These steps are saved in **Transform Sample File**.

Why?

Instead of repeating the cleaning process for every file manually, Power BI says:

For every file:



Apply

Transform Sample File

Conceptually:

January

↓

Clean

↓

Output

February

↓

Clean



Output

March



Clean



Output

The same transformation logic is reused for every workbook.

Query 4

Transform File (Function)

This query usually has an **fx** icon.

That means it is a **function**.

What is a Function?

A function takes an input, performs an operation, and returns an output.

Example in mathematics:

$$f(x)=x+2$$

If:

$$x=5$$

Output:

7

Power Query functions work similarly.

Input:

January.xlsx

Output:

Clean January Data

Input:

February.xlsx

Output:

Clean February Data

The function reuses the transformation steps defined in **Transform Sample File**.

Query 5

Parameter1

Many beginners ask:

What is Parameter1?

A parameter stores a value used by another query.

In this case, it usually stores the sample file reference.

Example:

Parameter1

↓

January.xlsx

Power BI uses this internally when building the sample transformation.

Most of the time, you don't need to modify it.

Step 4 – How Do These Queries Work Together?

Here's the complete flow:

Folder

|



List All Excel Files

|



Choose One Sample File

|



Learn Its Structure

|



Create Transformation Steps

|



Convert Steps into a Function

|



Apply Function to Every File

|



Append Results

|



One Combined Table

Notice that the helper queries each have a specific role.

Step 5 – Which Query Do You Use?

Most of the time, you'll work with the **main combined query** (often named after your folder or something like Transform File from Data).

You rarely edit:

- Sample File
- Parameter1
- Transform File (Function)

unless you have a specific reason.

Step 6 – Should You Delete Helper Queries?

No.

Deleting them can break the refresh process because the main query depends on them.

A common approach is:

- Keep the helper queries.
- Right-click each helper query and **disable load** (if they're being loaded unnecessarily).
- Rename the main query to something meaningful like:

FactCalls

This makes your data model easier to understand later.

Understanding Applied Steps

Look at the **Applied Steps** pane.

You might see:

Source

↓

Navigation



Promoted Headers



Changed Type

Each step is executed in order.

For example:

Source → Read the Excel file.



Navigation → Open the selected worksheet or table.



Promoted Headers → Convert the first row into column names.



Changed Type → Assign data types (Date, Text, Number, etc.).

If you delete one of these steps, later steps may fail because they depend on earlier ones.

Common Mistakes

- ✗ Deleting helper queries because they "look unused."
 - ✗ Editing the Sample File instead of the main query without understanding the impact.
 - ✗ Renaming or removing columns in only one monthly workbook.
 - ✗ Ignoring errors shown in Applied Steps.
-

Lesson 5 Checklist

Before moving to the next lesson, make sure you can answer these questions:

- ☒ What is Power Query?
 - ☒ Why does Power BI create a Sample File?
 - ☒ What is the purpose of Transform Sample File?
 - ☒ Why is Transform File a function?
 - ☒ What does Parameter1 store?
 - ☒ Which query will become your main dataset?
 - ☒ Why should helper queries generally not be deleted?
-

What Happens Next?

In **Lesson 6**, we'll begin cleaning the imported data in the main query.

We'll cover, step by step:

1. Understanding **Applied Steps** in detail.
2. Renaming the main query to FactCalls.
3. Promoting headers (if needed).

4. Setting correct data types.
5. Removing blank rows.
6. Removing duplicates.
7. Fixing inconsistent values (e.g., LAN, Lan, lan).
8. Reordering columns.
9. Adding custom columns (Month, Year, Quarter).
10. Preparing the data so it's ready for the Power BI data model.

This is where your raw Excel files start becoming a clean, professional dataset ready for dashboard development.

Excellent. Welcome to **Lesson 6**.

This is one of the **most important lessons** in the entire project.

80% of a Power BI project is data preparation.

Most beginners spend most of their time creating charts. Professional Power BI developers spend much more time making sure the data is clean and reliable before they build visuals.

Module 4 — Power Query Data Cleaning

Lesson 6 — Cleaning and Preparing Data

Objective

By the end of this lesson, you will know how to:

- Understand the **Applied Steps** pane.
- Rename the main query.
- Verify and correct data types.
- Remove blank rows.
- Remove duplicate records.

- Standardize inconsistent values.
 - Reorder columns.
 - Understand what Power Query does internally.
-

Step 1 – Open the Main Query

In the **Queries** pane on the left, you'll see several queries.

For example:

Queries

Folder Files

Sample File

Transform Sample File

Transform File

Parameter1

Click the **main combined query** (it may be named **Folder Files**, **Transform File from Data**, or similar).

Do not click:

- Sample File
- Parameter1
- Transform Sample File
- Transform File (function)

These are helper queries.

Why?

Think of it like a restaurant.

Kitchen Staff

|



Helper Queries

↓

Chef

↓

Main Query

↓

Customer

The dashboard only uses the **main query**.

Step 2 – Rename the Main Query

On the right side under **Query Settings**, you'll see:

Properties

Name

Folder Files

Rename it to:

FactCalls

Press **Enter**.

Why "FactCalls"?

In Business Intelligence, tables are generally divided into:

Fact Tables

Contain measurable business events.

Example:

Call

Ticket

Sales

Orders

Your dataset contains helpdesk tickets, so it is a **Fact Table**.

Dimension Tables

Contain descriptive information.

Example:

Department

Technician

Calendar

Problem Category

We'll create those later.

Step 3 – Understanding Applied Steps

On the right you'll see:

Applied Steps

Source

Navigation

Promoted Headers

Changed Type

These are not just labels.

They are **instructions** Power Query remembers.

Every time you click **Refresh**, Power BI executes them again.

Imagine your files contain:

January.xlsx

↓

February.xlsx

↓

March.xlsx

Power BI repeats:

Read File



Open Sheet



Use First Row as Headers



Assign Data Types



Append Rows

for every file automatically.

Think of Applied Steps Like a Recipe

Suppose you're making tea.

Boil Water

↓

Add Tea

↓

Add Milk

↓

Add Sugar

If you remove **Boil Water**, the later steps don't make sense.

The same is true in Power Query.

Every step depends on the previous one.

Step 4 – Check the Column Headers

Your table should look similar to:

| S.No | Date | Problem | User Name | Phone | Department | Action Taken |
Date of Completion | Status | Time Taken | Solved By |

Check:

- Are the column names correct?
 - Are there any blank headers?
 - Is the first row being used as headers?
-

If the headers look like this:

| Column1 | Column2 | Column3 |

Power BI did **not** recognize the first row as headers.

Fix it by:

Home → Use First Row as Headers

This creates the **Promoted Headers** step.

Step 5 – Verify Data Types

This is one of the most important tasks.

Every column should have the correct data type.

Look at the icon beside each column name.

Example:

Icon	Meaning
	Date
ABC	Text
123	Whole Number

Icon	Meaning
1.2	Decimal Number

For your dataset, use:

Column	Data Type
S.No	Whole Number
Date	Date
Problem	Text
User Name	Text
Phone	Whole Number (or Text if extensions can contain leading zeros or special characters)
Department	Text
Action Taken	Text
Date of Completion	Date
Status	Text
Time Taken	Whole Number
Solved By	Text

Why is this important?

Suppose:

Date

01-Jan-2024

is stored as **Text**.

Power BI cannot:

- Build a timeline correctly.

- Group by month or year.
- Perform time intelligence calculations.

Correct data types are essential for accurate analysis.

Step 6 – Remove Blank Rows

Sometimes Excel contains empty rows.

Example:

Date	Problem
01-Jan	Printer
<i>(blank)</i>	<i>(blank)</i>
02-Jan	LAN

These blank rows become useless records.

To remove them:

1. Select a column that should always contain data (for example, **S.No** or **Date**).
2. Filter out (null) values.

Alternatively:

Home → Remove Rows → Remove Blank Rows

(Note: this removes rows that are completely blank.)

Why?

Blank rows can:

- Increase row count.
- Affect measures.

- Create empty categories in visuals.
-

Step 7 – Remove Duplicates

Suppose:

S.No	Date	Problem
15	02-Jan	Printer
15	02-Jan	Printer

This may happen if someone copied data twice.

To remove duplicates:

1. Select the columns that define a unique call (for example, **S.No** if it is guaranteed unique).
2. Choose:

Home → Remove Rows → Remove Duplicates

Important

If **S.No** restarts every month (1–180 in every monthly file), then **S.No alone is NOT unique** after combining all months.

In that case, remove duplicates based on a combination such as:

- Date
- User Name
- Problem
- Department

or create a unique Ticket ID in the source data.

Step 8 – Standardize Text Values

Look for inconsistent spellings.

Example:

Original
LAN
Lan
lan

Power BI treats these as **three different categories**.

Standardize them using:

Transform → Format

Options include:

- Uppercase
- Lowercase
- Capitalize Each Word

Example:

LAN

becomes consistent across all records.

You can also use **Replace Values** to fix spelling variations.

Step 9 – Reorder Columns

Arrange the columns logically.

Recommended order:

Column
S.No

Column
Date
Problem
User Name
Phone
Department
Action Taken
Date of Completion
Status
Time Taken
Solved By

This doesn't affect calculations but makes the dataset easier to read and maintain.

What Happens Internally?

Every action you perform adds a new **Applied Step**.

For example:

Source



Navigation



Promoted Headers



Changed Type



Removed Blank Rows



Removed Duplicates



Reordered Columns

Power Query stores the transformation logic—not the transformed data itself.

Whenever you refresh:

New Excel File



Read File



Apply Every Step Again



Updated Clean Table

This is why you clean the data **once**, not every month.

Common Mistakes

- ✗ Leaving the Date column as Text.
 - ✗ Removing duplicates using the wrong column.
 - ✗ Ignoring inconsistent text values.
 - ✗ Renaming columns differently in one monthly workbook.
 - ✗ Editing data directly in Power BI instead of fixing recurring issues in Power Query.
-

Lesson 6 Checklist

Before moving on, confirm:

- ☒ Main query renamed to FactCalls.
- ☒ Headers are correct.
- ☒ Every column has the correct data type.
- ☒ Blank rows removed.

-  Duplicate records reviewed and removed if appropriate.
 -  Text values standardized.
 -  Columns arranged logically.
-

A Small Improvement to Your Dataset

Your current sample uses **S.No** that restarts every month (1–180).

In a real IT helpdesk, it's better to have a unique **Ticket ID**, such as:

HD-2024-000001

HD-2024-000002

HD-2024-000003

This makes it much easier to:

- Track individual tickets.
 - Prevent accidental duplicate removal.
 - Build drill-through reports.
 - Reference tickets in support workflows.
-

Next Lesson (Lesson 7)

Now that the data is clean, we'll **enrich** it by creating additional columns that make reporting much easier.

We'll learn:

1. Date vs. Auto Date Hierarchy.
2. Creating **Year**, **Quarter**, **Month Name**, **Month Number**, **Week Number**, and **Day Name**.
3. Why sorting Month Name by Month Number is necessary.

4. Creating a proper **Calendar (Date) table** and why professionals prefer it over relying on the raw Date column.

Perfect! Your Power Query setup looks **correct**. This is exactly what I expected after **Lesson 5**. Let's review your screen like we would in a real classroom.

What I See in Your Screenshot

1. Queries Pane (Left)

You have:

Transform File from Data

|

|— Helper Queries

| |— Parameter1

| |— Sample File

| |— Transform File

|

|— Transform Sample File

|

|— Data

 This is correct.

Power BI automatically created these helper queries.

Don't delete them.

The only query you'll normally work with is:

Data

Later we'll rename it to:

FactCalls

2. Main Query

You selected

Data

Excellent.

This is your main table.

3. Applied Steps

You currently have

Source

↓

Filtered Hidden Files1

↓

Invoke Custom Function1

↓

Renamed Columns1



Removed Other Columns1



Expanded Table Column1



Changed Type



Filtered Rows

Many beginners become scared seeing so many steps.

Actually, every one has a purpose.

Let's understand each one.

Step 1

Source

This is where Power BI remembers

D:\IT Helpdesk Dashboard\Data

Internally it stores something similar to

powerquery

Folder.Files("D:\IT Helpdesk Dashboard\Data")

Meaning

Read every file from this folder.

Step 2

Filtered Hidden Files

Question:

Why does Power BI create this automatically?

Suppose your folder contains

January.xlsx

February.xlsx

desktop.ini

~\$January.xlsx

Windows creates hidden files.

Power BI removes them automatically.

Otherwise refresh may fail.

Step 3

Invoke Custom Function

This is the most important step.

Remember our discussion?

Power BI created a function.

That function

Open Excel

↓

Read Table

↓

Clean Data

↓

Return Table

Now Power BI runs that function

for

January

↓

February

↓

March

↓

April

...

one by one.

Think of it like

for every Excel File

↓

Run Function

↓

Return Clean Table

Exactly the same concept as

Python

Run

for file in Folder:

clean(file)

Step 4

Renamed Columns

Power BI automatically renamed
columns like
Column1

Column2

to better names.

Usually you don't touch this.

Step 5

Removed Other Columns

Originally

Folder import contains

| Content | Name | Extension | Date Modified | Folder Path |

But you only need

Content

to open Excel.

Power BI removed unnecessary folder metadata.

Very good.

Step 6

Expanded Table Column

This is the magic step.

Before expansion

Power BI has

| File | Table |

Where

Table contains

Date

Problem

Department

...

After expanding

Power BI produces

| Date | Problem | Department |

This is why your dataset finally looks like a normal table.

Step 7

Changed Type

This assigns

Date

↓

Date Type

Problem

↓

Text

Time Taken

↓

Whole Number

Very important.

Step 8

Filtered Rows

This removes

null

rows.

Looks good.

Now Let's Check Your Dataset

From your screenshot

I noticed something.

Source.Name

The first column is

Source.Name

Example

2024_April.xlsx

Question:

Should we remove it?

Answer

No.

Keep it.

Why?

Later

we can create visuals like

Calls by File

January

February

March

Also

if refresh has errors

Source.Name tells us
which file caused the issue.
Very useful.

Your Date Column

Looks like

05-04-2024

I want you to check something.

Click

Date Column

Look at the icon.

Is it

 Date

or

ABC Text

?

If it is



Perfect.

If it is

ABC

We'll fix it.

Check Time Taken

Scroll right.

Check

Time Taken

Icon should be

123

Whole Number.

Check Status

Should be

ABC

Text.

Small Improvement

Rename

Data

to

FactCalls

How?

Right side

Properties

↓

Name



FactCalls

or

Right-click

Data



Rename

Type

FactCalls

This follows professional naming conventions.

Homework Before Lesson 7

Please verify these points:

Task	Status
Rename Data → FactCalls	☐
Check Date data type	☐

Task	Status
Check Time Taken data type	☐
Check Status data type	☐
Keep Source.Name column	☐

★ What We'll Learn in Lesson 7

This is where your dashboard becomes much more powerful.

We'll create a **professional Date Dimension (Calendar Table)** instead of relying only on the raw Date column.

You'll learn:

- Why Power BI professionals rarely use the raw Date column directly.
- How to create a proper Calendar table.
- Why we create **Year, Quarter, Month Number, Month Name, Week Number, and Day Name**.
- How relationships between FactCalls and the Calendar table work.
- Why visuals sort months incorrectly (April, August, December...) and how to fix it using **Sort by Column**.
- Time intelligence concepts that enable year-to-date, month-to-date, and trend analysis later in the project.

This is one of the biggest differences between a beginner Power BI report and a professional one.

Excellent. Your project is in a good state. Now we'll continue **Lesson 6** like a real Power BI developer. From this point onward, we won't just click buttons—we'll understand **why** we're doing each step.

Module 4 – Power Query Data Cleaning

Lesson 6 (Part 2) – Data Cleaning & Standardization

Current Situation

Your data looks like this:

Source Name	S.No	Date	Problem	User Name	Phone	Department	...
-------------	------	------	---------	-----------	-------	------------	-----

This is **raw business data**.

Our goal is to transform it into **analysis-ready data**.

Think of it like this:

Excel Files

|



Raw Data

|



Power Query Cleaning

|



Professional Dataset

|



Dashboard

Step 9 – Check Every Column

Don't assume Power BI detected everything correctly.

Go through each column one by one.

Column 1 – Source.Name

Current Value

2024_April.xlsx

Purpose

This tells us from which Excel file the row came.

Example

Source.Name	Problem
January.xlsx	Printer
February.xlsx	LAN
March.xlsx	VPN

Should we keep it?

Yes.

Many tutorials remove it immediately.

I don't recommend that.

Why?

Suppose one day your manager says:

"March report looks wrong."

How will you know which rows belong to March?

Simply filter

Source.Name

↓

2024_March.xlsx

Problem solved.

Step 10 – S.No

Current values

1

2

3

...

180

Question:

Should we keep S.No?

In a real company

No.

Because

January

1

2

3

February

1

2

3

Now duplicates exist.

S.No is no longer unique after combining months.

Better Solution

Real companies use

Ticket ID

HD-000001

HD-000002

HD-000003

Your sample data doesn't have Ticket IDs.

So for now

Keep S.No.

Later we can create a unique ID if needed.

Step 11 – Date Column

Click the Date column.

Check the icon.

It should show



Date

If it doesn't:

Change Data Type

1. Select **Date** column.
2. Go to:

Transform



Data Type



Date

What happens internally?

Power Query adds another Applied Step.

Something like

`powerquery`

`Table.TransformColumnTypes(...)`

Every refresh

Power BI remembers

Convert Date

↓

Always

No manual work again.

Why Date Type Matters

Imagine Date is Text.

Then

January

February

March

cannot be recognized properly for time-based analysis.

Many DAX functions also require a true Date type.

Step 12 – Phone (Extension)

Your column

1640

1418

1985

Question

Should this be Number?

Most beginners say

"Yes"

Actually

Not always.

Why?

Phone numbers are **identifiers**, not quantities.

You never calculate:

1640 + 1985

That has no business meaning.

Recommendation

Convert Phone (Extension) to **Text**.

Why?

- Prevents leading zeros from disappearing.
- Keeps identifiers intact.
- Makes joins more reliable.

To change it:

1. Select **Phone (Extension)**.
 2. **Transform** → **Data Type** → **Text**.
-

Step 13 – Problem Column

Look carefully.

Do you have values like:

LAN Issue

Lan issue

lan issue

or

Printer

Printer Issue

Printer problem

If yes

Power BI thinks they are different categories.

Standardization

Select the column.

Go to

Transform

↓

Format

Options

- Uppercase
 - Lowercase
 - Capitalize Each Word
 - Trim
 - Clean
-

Use These

Trim

Removes spaces.

Example

Printer

Printer

Leading/trailing spaces disappear.

Clean

Removes hidden characters.

Sometimes Excel imports

Printer ↵

Power BI removes invisible characters.

Capitalize Each Word

Transforms

printer issue

↓

Printer Issue

Professional appearance.

Step 14 – User Name

Current

User_001

User_250

User_178

Data type

Text

Correct.

Nothing to change.

Step 15 – Department

Look for spelling differences.

Example

Wrong

Department 1

Department1

department 1

Dept 1

Power BI creates four categories.

Instead

standardize them.

Replace Values

Go to

Transform

↓

Replace Values

Example

Replace

Dept 1

with

Department 1

Step 16 – Status

Your sample has

Completed

for every row.

Question

Should we keep it?

Yes.

Because later

your organization may add

Pending

In Progress

Completed

Cancelled

Your dashboard will already support it.

Step 17 – Time Taken

Current values

1

2

3

4

Make sure

Data Type

↓

Whole Number

Why?

Later

we'll calculate

Average Resolution Time

Maximum Resolution Time

Minimum Resolution Time

Those calculations require numeric data.

Step 18 – Check for Errors

Look for

Error

or

null

If any exist

Click the filter.

Remove

Errors

or

null

Step 19 – Review Applied Steps

After all your cleaning, the right pane might look something like:

Source

↓

Filtered Hidden Files

↓

Invoke Custom Function

↓

Expanded Table

↓

Promoted Headers

↓

Changed Type

↓

Trimmed Text

↓

Cleaned Text

↓

Replaced Values



Notice that Power Query has recorded every action.

You don't need to repeat them every month. On refresh, the same steps are automatically applied to any new Excel file added to the folder.

Step 20 – Save Your Work

Do not click Close & Apply yet if you're following along lesson by lesson.

Instead:

1. Press **Ctrl + S** to save your PBIX file.
2. Review the cleaned data.
3. Confirm there are no errors in the preview.

We'll use this clean query in the next lesson.

Best Practice (Professional Developers)

Professionals don't immediately load data into Power BI.

Before loading, they verify:

Check	Expected
Query Name	FactCalls
No Errors	✓
Correct Data Types	✓

Check	Expected
No Blank Rows	✓
Standardized Text	✓
Logical Column Order	✓
Applied Steps Clean	✓

Only after this checklist is complete do they click **Close & Apply**.

Homework Before Lesson 7

Please do these tasks:

- Rename **Data** → **FactCalls**.
 - Convert **Phone (Extension)** to **Text**.
 - Apply **Trim** and **Clean** to all text columns (Problem, User Name, Department, Action Taken, Status, Solved By).
 - Check that **Date** is a Date type.
 - Check that **Time Taken** is a Whole Number.
 - Save the PBIX file.
-

Next Lesson (Lesson 7)

We'll move beyond cleaning and start **enriching** the data.

You'll learn:

- Why professionals create a **Calendar (Date) Table**.
- How to create **Year, Quarter, Month Name, Month Number, Week Number, and Day Name**.
- Why Month Name sorts alphabetically by default and how to fix it.
- How a Calendar table enables powerful time-based analysis throughout your dashboard.

This is one of the most important modeling techniques in Power BI and a hallmark of professional reports.

Excellent! This is a **real-world Power BI error**, and it's actually a great learning opportunity. Don't worry—this is a common issue when working with folder imports.

Let's diagnose it.

Error Message

Expression.Error:

The key didn't match any rows in the table.

Details:

Key=

Item=Sheet1

Kind=Sheet

What does this mean?

Power BI is trying to open:

Sheet1

inside **every Excel file**.

But at least one of your Excel files **does not contain a worksheet named Sheet1**.

Why did this happen?

Suppose your files are:

January.xlsx

└─ Sheet1

February.xlsx

└─ Sheet1

March.xlsx

└─ Calls

April.xlsx

└─ Sheet1

When Power BI reaches **March.xlsx**, it searches for:

Sheet1

It doesn't exist.

So Power BI throws this error.

How do we verify this?

Step 1

Click the query:

Sample File

on the left.

Tell me:

- What is the **worksheet name**?
 - Is it:
 - o Sheet1
 - o Sheet
 - o Calls
 - o Table1
 - o Something else?
-

Step 2

Open **one of your Excel files** (for example 2024_April.xlsx).

Look at the bottom tabs.

Do you see:

Sheet1

or

January

or

Calls

I also noticed another issue in your screenshot

Your **Applied Steps** contain:

Filtered Rows

Filtered Rows1

Changed Type1

Transposed Table

 Why is Transposed Table there?

Did you click:

Transform

↓

Transpose

?

For your project, **you should NOT transpose the table.**

Your data is already in the correct orientation:

Rows = Calls

Columns = Fields

Transpose changes it to:

Rows = Columns

Columns = Rows

That is incorrect for this dataset.

First Fix

In the **Applied Steps** pane:

Click

Transposed Table

then click the  beside it.

Does the error disappear?

If the error still exists

Then the problem is definitely the worksheet name.

Most Likely Cause

When you imported the folder, Power BI learned from the **first sample file**.

Suppose that file contained:

Sheet1

Later, you renamed the worksheet in Excel to:

April

or

Calls

Now Power BI is still trying to find:

Sheet1

which no longer exists.

Please Answer These 3 Questions

1. Did you rename the worksheet in your Excel files after importing them into Power BI?
 2. Do all Excel files have exactly the same worksheet name?
 3. What is the worksheet name in one of your Excel files?
-

Don't continue to Lesson 7 yet.

We need to fix this import issue first, because if the folder import is broken, every later lesson (Calendar table, DAX, dashboard, etc.) will also break.

Once you answer the three questions (or share a screenshot of one Excel workbook showing the sheet tabs), I'll guide you through the exact fix step by step.

Good. This screenshot tells us a lot.

Your **Sample File** is correct.

I can see:

Name	Item	Kind
Sheet1	Sheet1	Sheet

So the sample file contains **Sheet1**, which is fine.

However, your previous screenshot showed this error:

The key didn't match any rows in the table

Item = Sheet1

Kind = Sheet

That means **at least one of the other Excel files does not have a sheet named Sheet1.**

First, let's verify which file is causing the problem.

Step 1

Click the query:

Transform Sample File

(not Sample File)

Take a screenshot and send it.

I want to see:

- Data preview
 - Applied Steps
 - Formula bar
-

Step 2

Open your **Data** folder in Windows Explorer.

Check every Excel file.

For example:

2024_January.xlsx

2024_February.xlsx

2024_March.xlsx

...

Open each one and verify the sheet tab at the bottom.

They **must all** be:

Sheet1

If one of them is:

January

or

Calls

or

Sheet2

that file is causing the error.

I Also Found Another Problem

From your earlier screenshot, I noticed this in **Applied Steps**:

Changed Type1

↓

Transposed Table

Those steps should **not** exist for this project.

That tells me you accidentally clicked **Transpose** while experimenting.

If that step still exists:

1. Click **Data** query.
2. In **Applied Steps**, delete:

Transposed Table

and also delete any unnecessary experimental steps such as Changed Type1 if they were added after the data was already correct.

Let's Find the Exact File

A very quick way is:

1. In Windows, open the **Data** folder.
2. Sort by **Name**.
3. Open each workbook.
4. Check the sheet name.

It should look like this:

2024_January.xlsx

└─ Sheet1

2024_February.xlsx

└─ Sheet1

2024_March.xlsx

└─ Sheet1

...

If even **one** workbook has a different sheet name, Power BI cannot combine them using the current transformation.

I Need One More Screenshot

Please send the **Transform Sample File** query (not the Sample File query).

That screenshot will let me pinpoint the exact failing step, and we'll fix it together before continuing the course. Once this is fixed, we'll proceed to **Lesson 7**.

Yes, that's exactly what happened. 👍

The error is **not because of your Excel files** anymore. It's because the **Transform Sample File** query has been accidentally modified.

From your screenshot, I can clearly see:

Expression.Error:

We cannot convert a value of type Table to type Binary.

and the formula bar shows:

powerquery

= Excel.Workbook(Parameter1, null, true)

This tells me **Parameter1 is no longer a Binary file**. It has become a **Table** because of the accidental transformation.

Don't worry

This is very common when learning Power Query.

We can fix it in 2–3 minutes.

Step 1

Go to

Transform Sample File

Look at **Applied Steps**.

Currently you have

Source

Navigation

Promoted Headers

Click the  beside

Promoted Headers

Does the error disappear?

If not,

Delete

Navigation

If it still doesn't,

Delete

Source

Do **not** worry.

We're not deleting data.

We're only deleting the transformation steps.

Step 2

Now click

Sample File

Does it still show

Sheet1

like your previous screenshot?

If yes,
good.

Step 3

Now click

Transform Sample File

Right-click it.

Choose

Delete

 **Don't do this yet** until I explain.

Normally we don't delete it.

But because it is corrupted, sometimes rebuilding it is the fastest solution.

Before deleting anything

I want to check one thing.

Click

Parameter1

Take a screenshot.

I want to see whether **Parameter1** contains

- Binary
- Table
- Excel Workbook

This will immediately tell me exactly where the corruption happened.

I have another recommendation

Since we are building this project as a course, I actually recommend something even better.

Your current project has become corrupted because of experimentation.

Instead of spending 20–30 minutes repairing helper queries, we can rebuild the folder connection in about **3 minutes**.

Since you now understand:

- Folder import
- Sample File
- Transform Sample File
- Functions
- Helper Queries

recreating them will reinforce the learning and give you a clean foundation for the rest of the course.

This is exactly what many Power BI developers do—if helper queries get corrupted, they simply recreate the folder connection rather than trying to repair every dependency.

My recommendation

1. **Don't delete your PBIX file.**
2. Save it as a backup.
3. Remove the current folder query.
4. Re-import the folder using **Get Data → Folder → Combine & Transform Data**.
5. Continue from Lesson 6 with a clean set of helper queries.

This is the quickest and most reliable path forward.

First, send me a screenshot of Parameter1. Once I confirm its contents, I'll tell you whether we can repair it in one step or whether recreating the folder connection is the better option.

Excellent. I can now identify the exact issue.

The error is **not in the Data query**. The Data query is failing because **Transform Sample File is corrupted**, and every other query depends on it.

The error:

Expression.Error:

We cannot convert a value of type Table to type Binary.

means that **Power BI expected a Binary (Excel file) but received a Table instead**.

Why did this happen?

You mentioned:

"I accidentally clicked on Transpose."

When you transposed the **Transform Sample File** query, you changed the type of data flowing through the helper queries.

The dependency chain is:

Parameter1 (Binary Excel File)

|



Transform Sample File

|



Transform File (Function)

|



Data

Once Transform Sample File became invalid, every query after it failed.

The Fastest and Professional Solution

Since we're still at the beginning of the project, **don't spend time repairing helper queries.**

Professional Power BI developers usually **recreate the folder connection** because it's faster and safer.

Step 1: Save a Backup

Press:

Ctrl + Shift + S

Save as:

Helpdesk_Backup.pbix

Step 2: Delete the Broken Queries

In Power Query, delete **all** these queries:

Data

Transform Sample File

Transform File

Sample File

Parameter1

or simply delete the top query:

Transform File from Data

Power BI will ask:

"This query has dependent queries. Delete all?"

Click:

Delete

This removes the corrupted helper queries together.

Step 3: Import Again

Go to:

Home

↓

New Source

↓

Folder

Choose:

IT Helpdesk Dashboard\Data

Click:

Combine & Transform Data

Power BI will recreate:

Data

Transform Sample File

Transform File

Sample File

Parameter1

Why is this the best solution?

Repairing helper queries requires editing **Power Query (M) code** and understanding query dependencies.

Recreating the import:

- Takes about 2 minutes.
 - Produces clean helper queries.
 - Matches exactly what Power BI expects.
 - Lets us continue the course without hidden issues.
-

Before Clicking Anything Like Transpose Again

A good habit is to ask:

"Am I transforming the main data, or am I inside a helper query?"

You should **never** perform transformations on:

- Sample File
- Transform Sample File
- Transform File
- Parameter1

All your cleaning (Trim, Replace Values, Data Types, Remove Rows, etc.) should be done only on the **main query** (Data, which we'll later rename to FactCalls).

What We'll Do Next

Once you've recreated the folder import:

1. Rename Data → FactCalls.
 2. Continue cleaning the data (Lesson 6).
 3. Click **Close & Apply**.
 4. Move to **Lesson 7: Building the Calendar Table and Data Model**.
-

One Suggestion

From this point onward, **don't experiment on the helper queries**. We'll use them only to understand how Power BI works. All transformations will be performed on the **main query (FactCalls)**. This mirrors how Power BI developers work in production projects and will help you avoid exactly this kind of issue in the future.

Excellent. Since you've recreated the queries (or are about to), let's continue **Lesson 6** properly. From now on, we'll follow **industry best practices**. I'll

explain **every click**, **why we do it**, **what Power BI does internally**, and **what to avoid**.

Module 4 – Data Cleaning with Power Query

Lesson 6 (Final)

Objective

At the end of this lesson, you'll have a **clean, professional dataset** ready for the data model.

Current Situation

After importing from the folder again, your query list should look like:

Transform File from Data

|

|— Helper Queries

| |— Parameter1

| |— Sample File

| |— Transform File

|

|— Transform Sample File

|

|— Data

Only **Data** is your working query.

Step 1 – Rename the Query

Why?

Imagine you have 20 queries.

Which one is easier to understand?

Data

or

FactCalls

Obviously

FactCalls

Professional developers always use meaningful names.

How?

Right-click

Data

↓

Rename

↓

Type

FactCalls

Press Enter.

Step 2 – Verify Column Names

Look at the first row.

It should be

| Source.Name | S.No | Date | Problem | User Name | Phone (Extension) |
Department | Action Taken | Date of Completion | Status | Time Taken | Solved
By |

Question:

Why?

Because Power BI uses these names later in

- DAX
- Relationships
- Charts
- Filters

If headers are wrong, everything becomes harder.

Step 3 – Verify Data Types

This is the first thing professionals do.

Look at the icon before every column.

Example

ABC

means

Text

123

means

Whole Number



means

Date

Your Dataset

Use these data types.

Column	Data Type	Why
Source.Name	Text	File name
S.No	Whole Number	Serial number
Date	Date	Time analysis
Problem	Text	Categories
User Name	Text	Names
Phone (Extension)	Text	Identifier
Department	Text	Categories
Action Taken	Text	Description
Date of Completion	Date	SLA
Status	Text	Completed/Pending
Time Taken	Whole Number	Average calculations
Solved By	Text	Technician

Why Phone Should Be Text

Many beginners keep it as Number.

Wrong.

Example

0015

becomes

15

You lose the leading zeros.

Phone numbers are identifiers, not values to calculate.

Step 4 – Remove Blank Rows

Go to

Home



Remove Rows



Remove Blank Rows

Why?

Excel users often leave empty rows.

Example

Date	Problem
Printer	Issue
<i>(blank)</i>	<i>(blank)</i>
LAN	Issue

Power BI counts that blank row.

Remove it.

Step 5 – Remove Duplicates

Suppose someone copied the same call twice.

Ticket	User
105	Ram
105	Ram

Remove duplicates.

Select

S.No

if unique, or better, use multiple columns if S.No repeats monthly.

Then

Home

↓

Remove Rows

↓

Remove Duplicates

Step 6 – Trim Text

One of the most important steps.

Select

Problem

Go to

Transform

↓

Format

↓

Trim

What is Trim?

Suppose Excel contains

Printer Issue

with hidden spaces.

Trim removes spaces before and after the text.

Example

" Printer Issue "

↓

"Printer Issue"

Apply Trim To

- Problem
 - User Name
 - Department
 - Action Taken
 - Status
 - Solved By
-

Step 7 – Clean Text

Go to

Transform



Format



Clean

What is Clean?

Sometimes Excel contains invisible characters copied from emails or other systems.

Example

Printer ↵

Looks normal

Actually contains

Printer + hidden character

Clean removes these invisible characters.

Step 8 – Capitalize Properly

Go to

Transform

↓

Format

↓

Capitalize Each Word

Example

printer issue



Printer Issue

Much better for dashboards.

Step 9 – Standardize Values

Suppose your data contains

LAN Issue

Lan Issue

lan issue

Power BI treats them as three different categories.

Replace them.

Transform



Replace Values

Example

Replace

Lan Issue

with

LAN Issue

Step 10 – Review Applied Steps

Now your Applied Steps should look something like

Source



Filtered Hidden Files



Invoke Custom Function



Expanded Table



Changed Type



Removed Blank Rows



Removed Duplicates



Trimmed Text

↓

Cleaned Text

↓

Every step is recorded.

Nothing is permanently changing your Excel files. Power Query simply remembers these transformations and reapplies them whenever you refresh.

Step 11 – Check for Errors

Look carefully.

Do you see

Error

or

null

in any column?

If yes,

filter them out or fix the source data.

Step 12 – Rename Columns (Optional)

For cleaner reports, you can shorten names slightly.

Example:

Current	Better
Phone (Extension)	Extension
Date of Completion	Completion Date
Time Taken	Resolution Hours
Solved By	Technician

This makes report fields easier to read, but keep names meaningful.

Step 13 – Reorder Columns

Arrange them logically:

Order
Source.Name
S.No
Date
Problem
User Name
Extension
Department
Action Taken
Completion Date
Status
Resolution Hours
Technician

Step 14 – Verify Row Count

Look at the bottom left of Power Query.

Suppose you imported

12 files

×

180 rows

Expected

2160 rows

If you expected 2160 but only see 180,
then only one file was imported.

This is an important validation step.

Step 15 – Save Your Work

Press

Ctrl + S

Step 16 – Close & Apply

Now click

Home

↓

Close & Apply

What happens internally?

Power BI:

1. Executes every Applied Step.
2. Creates the final FactCalls table.
3. Loads it into the Data Model.
4. Compresses the data.
5. Makes it available for visuals and DAX.

Think of it like this:

Excel Files

|



Power Query

(Cleaning)

|



FactCalls

|



Power BI Data Model

Lesson 6 Checklist

Before moving on, confirm:

- ☒ Query renamed to FactCalls
- ☒ Correct data types

-  Blank rows removed
 -  Duplicate records checked
 -  Trim applied to text columns
 -  Clean applied to text columns
 -  Standardized category values
 -  Columns reordered
 -  Close & Apply completed successfully
-

End of Lesson 6

Congratulations! You have now completed the **data preparation phase**, which is often the largest part of a Power BI project.

Next Lesson (Lesson 7)

In **Lesson 7**, we'll move into **Data Modeling**, where you'll learn:

- Why professional Power BI developers create a **Calendar (Date) table** instead of relying on the raw Date column.
- How to build a Calendar table.
- How to create relationships between FactCalls and the Calendar table.
- Why Month Name sorts alphabetically by default and how to fix it.
- The foundation for time intelligence (Year-to-Date, Month-to-Date, trend analysis, etc.).

This is one of the biggest milestones in building a professional Power BI dashboard.

Excellent question. This is exactly how a **Power BI developer** thinks.

The answer is:

You should NOT remove duplicates just because Power Query has that option. First, you must understand what defines a duplicate in your business.

Let's use your IT Helpdesk dataset.

What is a Duplicate?

A duplicate means **the same business record appears more than once.**

Example:

S.No	Date	User	Problem	Department
15	05-Apr-2024	User_100	Printer Issue	HR
15	05-Apr-2024	User_100	Printer Issue	HR

These two rows are identical.

One of them can be removed.

In Your Dataset

Your columns are:

| Source.Name | S.No | Date | Problem | User Name | Extension | Department |
Action Taken | Completion Date | Status | Time Taken | Solved By |

Notice something important.

S.No is NOT Unique

January:

S.No
1

S.No
2
3

February:

S.No
1
2
3

So after combining files:

Source.Name	S.No
January.xlsx	1
February.xlsx	1

These are **not duplicates** because they belong to different files.

Should We Remove Duplicates?

Answer

No.

Because your dataset doesn't have a true unique identifier like:

Ticket ID

In Real Companies

The data looks like:

Ticket ID	Date	User
HD000001	01-Jan	User1

Ticket ID	Date	User
HD000002	01-Jan	User2

Here, removing duplicates is easy because **Ticket ID is unique**.

Your Sample Dataset

Suppose two users report the same issue.

Date	Problem	User
05-Apr	Printer Issue	User_001
05-Apr	Printer Issue	User_002

Are these duplicates?

 No.

Different users created different tickets.

Suppose

Date	Problem	User
05-Apr	Printer Issue	User_001
05-Apr	Printer Issue	User_001

Still not enough information.

Maybe the same user raised two separate tickets.

So we **cannot automatically delete one**.

How Professionals Decide

Before removing duplicates, they ask the business:

"What makes one helpdesk ticket unique?"

Possible answers:

- Ticket ID
- Incident Number
- Complaint Number
- Service Request Number

Without one of these, removing duplicates can accidentally delete valid records.

So What Should You Do in This Project?

Option 1 (Recommended)

Don't remove duplicates.

Reason:

Your sample data doesn't have a unique ticket identifier.

This is the safest and most correct choice.

Option 2

Create a new unique ID.

Example:

Source.Name	S.No	TicketID
January.xlsx	1	January-1
January.xlsx	2	January-2
February.xlsx	1	February-1

or

TicketID
HD-2024-000001
HD-2024-000002

Now every row is uniquely identifiable.

If You Really Want to Remove Exact Duplicates

Suppose someone accidentally copied the **same row twice**.

Then:

1. Select **all columns** (Ctrl + A in the preview or click the top-left table selector).
2. Go to:

Home

↓

Remove Rows

↓

Remove Duplicates

Power Query will remove a row **only if every selected column matches exactly**.

This is much safer than selecting only S.No.

Best Practice for Your Project

Since we're building this as a **professional IT Helpdesk dashboard**, I recommend:

-  Do **not** remove duplicates based on S.No.
-  Do **not** remove duplicates based on Date or User Name.

- ☒ Keep all rows.
 - ☒ Add a proper Ticket ID column in future datasets if possible.
 - ☒ Only remove **fully identical rows** if you know they resulted from accidental duplication.
-

My recommendation for our course

For **this dataset**, skip the **"Remove Duplicates" step** because there is no reliable business key to identify duplicates.

This is actually how an experienced Power BI developer would handle the situation: **never delete data unless you're certain it's a duplicate.**

Now we can safely move to **Lesson 7 – Creating the Calendar (Date) Table and Building the Data Model**, which is the next major milestone in the project.

Excellent question. This is actually one of the **most misunderstood features** in Power Query.

The dialog you're showing is **not meant to find all similar values automatically**. It replaces **one specific value** with another.

Let's understand it properly.

What does "Replace Values" do?

Suppose your **Problem** column contains:

Problem
LAN Issue
Lan Issue
lan issue
LAN issue
Printer Issue

You want all of them to become:

LAN Issue

You use **Replace Values**.

Example 1

Select the **Problem** column.

Go to:

Transform

↓

Replace Values

The dialog asks:

Value To Find

Lan Issue

Replace With

LAN Issue

Click **OK**.

Power Query changes **every occurrence** of:

Lan Issue

to

LAN Issue

It doesn't just change the selected row—it changes every matching value in the selected column.

Example 2

Suppose your data is:

Department
Dept 1
Dept 1
Dept 1
Department 1

Replace

Dept 1

with

Department 1

Result

Department
Department 1
Department 1
Department 1
Department 1

But how do I know what values exist?

This is the important part.

Professionals **never guess**.

Method 1 (Recommended)

Click the filter arrow (▼) on the column.

Example:

Problem ▼

You'll see

LAN Issue

Lan Issue

lan issue

Printer Issue

VPN Issue

...

Now you immediately know what inconsistent values exist.

Then replace them one by one.

Method 2 (Best for Large Datasets)

Instead of looking at thousands of rows,

Right-click

Problem

↓

Duplicate Column

Rename it

Problem_Check

Now

Home



Keep Rows



Keep Duplicates

or

Remove Duplicates

to create a list of unique values for inspection.

Even easier:

Select the column and use the filter dropdown to inspect the unique values.

Can Power Query automatically identify similar words?

Example

LAN Issue

Lan Issue

lan issue

LAN Issues

LAN Problem

Answer

No.

Power Query does **not** understand language or meaning.

It performs **exact text matching**.

For example:

LAN Issue

=/

LAN Issues

because one has an extra **s**.

Then how do professionals standardize data?

Step 1

Apply

Transform

↓

Format



Trim

Removes extra spaces.

Step 2

Apply

Transform



Format



Clean

Removes hidden characters.

Step 3

Apply

Transform



Format



Capitalize Each Word

Now

lan issue



Lan Issue

Step 4

Now use

Replace Values

Only for the remaining inconsistencies.

Example

Lan Issue

↓

LAN Issue

Best Method (Used in Companies)

Instead of repeatedly using **Replace Values**, create a **Lookup (Mapping) Table**.

Example:

Original Value	Standard Value
Lan Issue	LAN Issue
lan issue	LAN Issue
LAN issue	LAN Issue
Printer problem	Printer Issue
Printer Error	Printer Issue

Then use **Merge Queries** to automatically standardize the data.

Advantages:

- Easy to maintain.
- New synonyms can be added without editing transformation steps.
- Preferred for large datasets.

We'll learn this technique later in the course.

For Your Project

Looking at your generated data, I noticed the values are already consistent:

Monitor Display Issue

Windows Update Issue

System Not Booting

Email Issue

VPN Issue

Since the dataset was generated with standardized values, you **do not need to use Replace Values** at this stage.

You can simply:

-  Apply **Trim**
 -  Apply **Clean**
 -  Apply **Capitalize Each Word** (if needed)
 -  Skip **Replace Values** unless you actually find inconsistent spellings.
-

Professional Workflow

Import Data

|



Trim

|



Clean

|



Capitalize



Review Unique Values



Replace Values (only if needed)



Load Data

This minimizes unnecessary transformation steps and keeps your Power Query easy to maintain.

After completing these cleaning steps, we'll move to **Lesson 7**, where you'll build a proper **Calendar (Date) Table** and start creating the data model that powers professional Power BI reports.

Excellent! 🎉

You've completed one of the most important phases of a Power BI project: **Data Preparation**.

Now we move to the phase that separates **beginners** from **professional Power BI developers**.

Module 5 – Data Modeling

Lesson 7 – Creating a Calendar (Date) Table

Lesson Objective

By the end of this lesson, you'll understand:

- Why professionals create a Calendar table.
 - Why **Auto Date/Time** is not enough.
 - How to create a Calendar table.
 - How to create relationships.
 - Why Month Name sorts incorrectly.
 - How Power BI performs time intelligence.
-

What is a Data Model?

Imagine your current data.

You have only one table.

FactCalls

It contains

Date	Department	Problem	Technician
------	------------	---------	------------

Beginners build reports directly from this table.

It works...

until the project becomes bigger.

Real Company Example

Suppose after 6 months your manager asks:

Show me

- Monthly Calls

- Quarterly Calls
- Yearly Calls
- Weekend Calls
- Working Days
- Financial Year
- Holiday Analysis

Does your table have

Quarter

?

No.

Does it have

Week Number

?

No.

Does it have

Day Name

?

No.

Now you'll have to create these repeatedly.

Professionals don't do this.

Professional Architecture

Instead of

FactCalls

alone,
they create

Calendar

Like this

Calendar

|

|



FactCalls _____

What is a Calendar Table?

Think of it as

one row for every date.

Example

Date
01-Jan-2024
02-Jan-2024
03-Jan-2024
04-Jan-2024
...
31-Dec-2025

Notice

Even if

no calls happened

on

03-Jan

the date still exists.

That's extremely important.

Why?

Suppose

there were

No Calls

on Sunday.

If Sunday doesn't exist,

Power BI cannot show

0 Calls

Instead,

Sunday disappears.

Professional reports always show

Sunday

0 Calls

Why Not Use the Date Column?

Many beginners ask

"I already have a Date column."

Correct.

But that column only contains dates where calls exist.

Example

Date
01-Jan
03-Jan
04-Jan

Notice

02-Jan

is missing.

Power BI cannot analyze missing dates properly.

Calendar table solves this.

Calendar Table Contains Much More

It doesn't just contain

Date.

It also contains

| Date | Year | Quarter | Month | Month No | Week | Day | Day No |

Example

Date	Year	Quarter	Month	Month No	Day
05-Apr-2024	2024	Q2	April	4	Friday

Now every report becomes easy.

Where Do We Create It?

Not in Power Query.

Professionals create it in

Data View

using

DAX

Why?

Because

Power Query

↓

Data Cleaning

DAX

↓

Data Modeling

Different responsibilities.

Step 1

Close Power Query

Home

↓

Close & Apply

Wait until data loads.

Step 2

Go to

Data View

Left side

You'll see

Report



Data



Model

Click

Data

Step 3

Go to

Modeling

↓

New Table

Very important.

Not

New Column

Not

New Measure

Choose

New Table

Formula Bar Appears

You'll see

Table =

Delete it.

Enter This Formula

DAX

Calendar =

CALENDAR(

```
MIN(FactCalls[Date]),  
MAX(FactCalls[Date])  
)
```

Press

Enter

What Does This Mean?

Let's break it down.

CALENDAR()

creates dates.

Inside

MIN()

means

Find

the earliest call date.

Suppose

01-Jan-2024

MAX()

means

Find

the latest call date.

Suppose

31-Dec-2024

Power BI creates
every date
between
01-Jan

↓

31-Dec
Automatically.

Result

New Table
Calendar
Contains

Date
01-Jan
02-Jan
03-Jan
...

365 rows.

Why is this Dynamic?

Suppose next year

you add

2025_January.xlsx

Latest date becomes

31-Dec-2025

When you click

Refresh

Calendar automatically extends.

No editing required.

Verify It

Click

Calendar

Do you see

only

Date

column?

Good.

That's exactly what we want.

What Happens Internally?

FactCalls

↓

Earliest Date

↓

Latest Date

↓

Generate Every Date

↓

Store in Calendar Table

Why This Is Better Than Typing Dates

Some beginners manually type

01-Jan

02-Jan

03-Jan

...

Never do that.

Dynamic Calendar

updates automatically.

Homework Before Lesson 8

Complete these steps:

- ✓ Close & Apply.
- ✓ Go to **Data View**.
- ✓ Click **Modeling** → **New Table**.
- ✓ Create:

DAX

Calendar =

CALENDAR(

MIN(FactCalls[Date]),

MAX(FactCalls[Date])

)

✓ Confirm that the Calendar table appears with a single Date column.

What We'll Learn in Lesson 8

Right now, the Calendar table has only one column:

| Date |

We'll make it much more powerful by adding:

- Year
- Quarter
- Month Name
- Month Number
- Week Number
- Day Name
- Day Number
- Weekend/Weekday flag

This transforms it into a professional Date Dimension that you'll reuse in almost every Power BI project.

Excellent! 👍

You're at the **Folder Preview** screen. This is exactly where every professional Power BI project begins.

Let's continue **Lesson 7**, but first let's finish the import correctly.

Current Screen Explained

You are seeing:

Preview Folder Data

Power BI has **NOT imported your Excel data yet.**

It has only read the **folder contents.**

Notice the columns:

Column	Meaning
Content	Binary representation of each Excel file
Name	Excel file name
Extension	File extension (.xlsx)
Date Modified	Last modified date
Folder Path	File location

Think of it as Windows Explorer inside Power BI.

What Should You Click?

At the bottom-right you have

Cancel

Combine

Load ▼

Don't click **Load**

If you click **Load**, Power BI imports only this table:

| Name | Extension | Date Modified |

It **does not** read the Excel data.

That's not what we want.

Click **Combine**

Click the small arrow beside **Combine**.

You'll see:

Combine & Load

Combine & Transform Data

Choose

Combine & Transform Data

Why?

Because we still want to:

- Clean data
- Change data types
- Rename columns
- Standardize text
- Remove blanks

Professionals almost always choose **Combine & Transform Data**.

What Happens After Clicking?

Power BI opens a dialog like:

Combine Files

It asks which object to import.

Example:

Sheet1

Table1

Calls

Choose **Sheet1** (or the table if you formatted your data as an Excel Table).

Click **OK**.

Power BI Will Automatically Create

Queries

Transform File from Data

|

|— Helper Queries

| |— Parameter1

| |— Sample File

| |— Transform File

|

|— Transform Sample File

|

|— Data

Don't worry about these helper queries.

Only work on **Data**.

After Power Query Opens

Do these steps **in order**.

Step 1

Rename

Data

to

FactCalls

Step 2

Verify data types.

Use:

Column	Type
Date	Date
Problem	Text
User Name	Text
Phone (Extension)	Text
Department	Text
Action Taken	Text
Completion Date	Date
Status	Text
Time Taken	Whole Number
Solved By	Text

Step 3

Apply **Trim** to these columns:

- Problem
- User Name
- Department
- Action Taken
- Status
- Solved By

Go to:

Transform

↓

Format

↓

Trim

Step 4

Apply **Clean** to the same text columns:

Transform

↓

Format

↓

Clean

Step 5

Review your data.

Since this dataset is generated and already standardized, you can skip **Replace Values** unless you actually find inconsistent values.

Step 6

Click:

Home



Close & Apply

Wait until Power BI loads the data.

We Are Still in Lesson 7

Notice something important.

We have **not created the Calendar table yet**.

That comes **after** the data is loaded into the model.

Lesson 7 Roadmap

Once **Close & Apply** finishes, we'll continue with:

Part 2 – Data Modeling

We'll learn:

1. What is a Star Schema?
2. Why professionals create a Calendar table.
3. Create the Calendar table using DAX.
4. Create relationships.
5. Understand one-to-many relationships.
6. Why Month Name sorts alphabetically.

7. Fix Month sorting.
 8. Build the foundation for all future dashboards.
-

✓ Your Next Action

Please:

1. Click **Combine → Combine & Transform Data**.
2. Complete the cleaning steps.
3. Click **Close & Apply**.
4. Wait until the data loads.
5. Send me a screenshot of the **Power BI main window** (Report/Data/Model view) after the loading completes.

Then we'll begin the **actual Data Modeling** phase, which is where professional Power BI development really starts.

Excellent! 🎉

Your data has been successfully loaded into Power BI.

I can see:

- ✓ FactCalls table loaded
- ✓ No Power Query errors
- ✓ Report view is open
- ✓ Ready for Data Modeling

Now we officially begin **Module 5 – Data Modeling**.

Module 5 – Data Modeling

Lesson 7 – Creating the Calendar (Date) Table

What is Data Modeling?

Before clicking anything, understand the purpose.

Currently, your project looks like this:

FactCalls

Date

Problem

Department

User

Technician

Status

...

Everything is inside one table.

For small reports, that's okay.

For professional reports, it becomes difficult.

Why Create a Calendar Table?

Suppose your manager asks:

"Show me"

- Calls in January
- Calls in Quarter 2
- Calls on Mondays
- Calls on Weekends
- Month-over-Month Growth
- Year-to-Date Calls

Does your table contain

Quarter

?

No.

Does it contain

Month Number

?

No.

Does it contain

Week Number

?

No.

A Calendar table solves all of this.

Professional Data Model

Instead of

FactCalls

only,

professionals build

Calendar

|

|



FactCalls

Later

Department

|

Technician

Problem Category

Calendar

↓

FactCalls

This is called a **Star Schema**.

We'll build it gradually.

STEP 1 – Go to Data View

Look at the left side.

You'll see three icons.

 Report

 Data

 Model

Currently you're here:

 Report View

Click

 Data

What Happens?

Now you'll see your table like Excel.

Example

Date	Problem	User
05-Apr	Printer	User1

This lets us inspect the loaded data.

STEP 2 – Open Modeling Tab

At the top ribbon click

Modeling

You'll see buttons like

New Measure

New Column

New Table

Question

Which one do we choose?

Not

 New Measure

Not

 New Column

Choose

 **New Table**

Why New Table?

Think about it.

We want

Calendar

which is

an entirely new table.

Not

a column.

Not

a calculation.

STEP 3 – Click New Table

Click

Modeling

↓

New Table

Immediately

Power BI shows

Table =

in the formula bar.

What is DAX?

Everything you write here is

DAX

(Data Analysis Expressions)

Think of it like

Python

↓

for Machine Learning

SQL

↓

for Databases

DAX

↓

for Power BI

STEP 4 – Type This Formula

Delete

Table =

Type

DAX

Calendar =

CALENDAR(

MIN(FactCalls[Date]),

MAX(FactCalls[Date])

)

Don't press Enter yet.

Let's understand it.

Understanding CALENDAR()

Imagine your data.

Earliest Date

01-Jan-2024

Latest Date

31-Dec-2024

Power BI reads

MIN(FactCalls[Date])

↓

find earliest date

and

MAX(FactCalls[Date])

↓

find latest date

Then

creates

every date

between them.

Internally

Power BI does something like

Start

↓

01-Jan-2024

↓

02-Jan-2024

↓

03-Jan-2024

↓

...

↓

31-Dec-2024

Automatically.

Press ENTER

Power BI creates

Calendar

table.

Look at the Fields pane.

You should now see

FactCalls

Calendar

Verify the Calendar Table

Click

Calendar

You should see

Date
01-Jan-2024
02-Jan-2024
03-Jan-2024

...

Up to

31-Dec-2024

One row for every day.

Why is This Better?

Suppose your helpdesk had no calls on Sunday.

Without a Calendar table:

Sunday

✗ Missing

With a Calendar table:

Sunday

0 Calls

This is extremely important for accurate charts.

Dynamic Behavior

Suppose next month you add

2025_January.xlsx

and refresh.

The Calendar automatically extends to include the new dates because it always uses:

DAX

MIN(FactCalls[Date])

MAX(FactCalls[Date])

You don't need to edit the formula.

Your Homework

Complete these steps:

1. Go to **Data View**.
2. Click **Modeling** → **New Table**.
3. Create:

DAX

Calendar =

```
CALENDAR(  
MIN(FactCalls[Date]),  
MAX(FactCalls[Date])  
)
```

4. Press **Enter**.
 5. Confirm that the **Calendar** table appears.
-

Stop Here

Don't create relationships yet.

That is **Lesson 8**.

In Lesson 8, we'll transform this simple Calendar table into a **professional Date Dimension** by adding:

- Year
- Quarter
- Month Name
- Month Number

- Week Number
- Day Name
- Day Number
- Weekend/Weekday flag

Then we'll create the relationship:

Calendar[Date]

|

| 1 → *



FactCalls[Date]

This relationship is the foundation for all time-based analysis in your dashboard.

Excellent observation! 🙌

You're using the **new Power BI Desktop (2025/2026 UI)**. Microsoft has renamed some views.

In the older Power BI versions, the left pane was:



Report View



Data View



Model View

In your version, it is:



Report View



Table View □ (This replaces Data View)



Model View

DAX View

TMDL View

So whenever I say **Data View**, in your version use **Table View**.

Updated Lesson 7 (Power BI 2026 UI)

Step 1

Click

 Table View

(left sidebar)

You should see something like:

FactCalls

Date

Problem

Department

...

This is exactly what **Data View** was in older versions.

=====

Step 2

Click the **Modeling** tab at the top.

Look for:

New Measure

New Column

New Table

Click:

New Table

=====

Step 3

A formula bar appears.

Type:

DAX

Calendar =

CALENDAR(

MIN(FactCalls[Date]),

MAX(FactCalls[Date])

)

Press **Enter**.

What Happens?

A new table appears:

FactCalls

Calendar

Click **Calendar**.

You should see:

Date
01-01-2024
02-01-2024
03-01-2024
...

Why Microsoft Changed It

Microsoft introduced new developer-oriented views:

Old Power BI	New Power BI
Report View	Report View
Data View	Table View
Model View	Model View
—	DAX View
—	TMDL View

DAX View

A dedicated editor for writing and managing DAX formulas.

TMDL View

Used by advanced developers to edit the **Tabular Model Definition Language (TMDL)**, which represents the semantic model as text. Most report developers don't need to use it.

Which Views Will We Use in This Course?

We'll mainly work with these:

View	Purpose	Use Often?
 Report View	Build dashboards	✓ Very Often
 Table View	Inspect tables and data	✓ Often
 Model View	Create relationships	✓ Often
DAX View	Advanced DAX editing	◆ Occasionally
TMDL View	Advanced model development	✗ Not needed for this project

Before We Continue

Can you tell me:

When you click the Modeling tab, do you see a New Table button?

If you're unsure, share a screenshot of the **Modeling** ribbon, and I'll point out the exact button.

Perfect! 

This is **exactly what should happen**.

Your Calendar table has been created successfully.

The values:

Date
01-01-2024 00:00:00
02-01-2024 00:00:00
03-01-2024 00:00:00

are **DateTime** values.

Why is 00:00:00 Showing?

A DateTime value contains two parts:

Date + Time

Example:

01-01-2024 00:00:00

means

- Date = **01-Jan-2024**
- Time = **12:00:00 AM (midnight)**

Since your Calendar was created using `CALENDAR()`, every date defaults to midnight (00:00:00).

There is **nothing wrong**.

But We Don't Need the Time

For reporting, we only need:

01-01-2024

not

01-01-2024 00:00:00

Let's fix it.

Step 1

Select the **Date** column in the Calendar table.

Step 2

At the top, click **Column tools** (or **Table tools**, depending on your version).

You should see:

Data type

Change it to:

Date

(not **Date/Time**).

Step 3

Now look for:

Format

Choose a format such as:

dd-MM-yyyy

or

dd-MMM-yyyy

Example:

01-Jan-2024

Now the time (00:00:00) disappears from the display.

Why Is This Important?

If you keep it as **DateTime**:

- Relationships may still work, but matching Date-only fields can be confusing if the fact table also contains times.
 - Time intelligence is cleaner when you're working with **Date** values for a date dimension.
-

Verify Your FactCalls Date Column

Before we continue, check the **FactCalls** table.

Select:

FactCalls

↓

Click the **Date** column.

Is its **Data type**:

- ☒ **Date**

- or
-  **Date/Time**

Ideally, both:

FactCalls[Date]

and

Calendar[Date]

should be **Date**.

Great Progress!

So far you've completed:

-  Imported all monthly Excel files
 -  Cleaned the data
 -  Created the FactCalls table
 -  Created the Calendar table
-

Next Lesson (Lesson 8)

We'll transform this simple Calendar table into a **professional Date Dimension** by adding:

New Column	Example
Year	2024
Quarter	Q1
Month Number	1
Month Name	January
Week Number	1
Day Name	Monday

New Column	Example
Day Number	1
Is Weekend	Yes / No

These columns will make it easy to build monthly, quarterly, yearly, and weekday analyses without writing repetitive formulas.

This is the same approach used in enterprise Power BI projects.

Excellent! 🎉

You've successfully created your **Calendar** table.

Now we move to the **most important part of Lesson 7**.

This is where beginners stop, but professional Power BI developers continue.

Module 5 – Data Modeling

Lesson 7 (Part 2) – Building a Professional Calendar Table

Current Situation

Your Calendar table has only one column.

Date
01-Jan-2024
02-Jan-2024
03-Jan-2024

This is good...

But not enough.

Our Goal

We want to convert it into this:

Date	Year	Quarter	Month No	Month Name	Week No	Day No	Day Name	Week end
01-Jan-2024	2024	Q1	1	January	1	1	Monday	No

This is called a **Date Dimension**.

Why?

Imagine your manager asks:

 Show monthly calls.

Without Month Name,

Power BI cannot easily group months.

Imagine asking

 Show Quarter-wise Calls.

Without Quarter,

you'll have to calculate it every time.

Professional developers calculate it **once** in the Calendar table.

STEP 1 – Create Year Column

Go to

Table View

Select

Calendar

Click

Modeling

↓

New Column

Formula:

DAX

Year = YEAR(Calendar[Date])

Press **Enter**.

What happens?

Power BI reads

01-Jan-2024

↓

Extracts

2024

Result

Date	Year
01-Jan	2024
02-Jan	2024

STEP 2 – Create Quarter

Click

New Column

Formula

DAX

Quarter = "Q" & QUARTER(Calendar[Date])

Press Enter.

Result

Date	Quarter
01-Jan	Q1
05-Apr	Q2
12-Aug	Q3

STEP 3 – Month Number

Click

New Column

Formula

DAX

Month Number = MONTH(Calendar[Date])

Result

Date	Month Number
01-Jan	1
15-Feb	2
20-Dec	12

STEP 4 – Month Name

Click

New Column

Formula

DAX

Month Name = FORMAT(Calendar[Date], "MMMM")

Result

Date	Month Name
01-Jan	January
02-Feb	February

Very Important

Most beginners stop here.

But now your months will appear like this:

April

August

December

February

January

Why?

Because Power BI sorts text alphabetically.

We'll fix that shortly.

STEP 5 – Day Number

New Column

DAX

Day Number = DAY(Calendar[Date])

Result

Date	Day Number
05-Apr	5

STEP 6 – Day Name

New Column

DAX

Day Name = FORMAT(Calendar[Date], "dddd")

Result

Date	Day Name
01-Jan	Monday
02-Jan	Tuesday

STEP 7 – Week Number

New Column

DAX

Week Number = WEEKNUM(Calendar[Date])

Result

Date	Week Number
01-Jan	1
15-Jan	3

STEP 8 – Weekend Flag

New Column

DAX

Is Weekend =

IF(

WEEKDAY(Calendar[Date],2)>5,

"Yes",

"No"

)

How does this work?

WEEKDAY(...,2) returns:

Day	Value
Monday	1
Tuesday	2
Wednesday	3

Day	Value
Thursday	4
Friday	5
Saturday	6
Sunday	7

So

DAX

>5

means

Saturday

Sunday

↓

Weekend

Result

Date	Is Weekend
06-Jan	Yes
07-Jan	Yes
08-Jan	No

STEP 9 – Fix Month Sorting ★★★★★

This is one of the most important steps.

Currently

Month Name

is text.

Power BI sorts

April

August

December

February

Wrong.

Fix

Select

Month Name

Go to

Column tools

↓

Click

Sort by Column

↓

Choose

Month Number

Now

Power BI always shows

January

February

March

April

May

June

July

August

September

October

November

December

Correct!

What You Should Have Now

Your Calendar table should contain:

Column
Date
Year
Quarter
Month Number
Month Name
Day Number
Day Name
Week Number
Is Weekend

Why We Create All These Columns

Column	Used For
Year	Yearly Reports
Quarter	Quarter Analysis
Month Number	Sorting
Month Name	Charts & Slicers

Column	Used For
Week Number	Weekly Analysis
Day Name	Weekday Analysis
Is Weekend	Weekend vs Weekday Reports

End of Lesson 7

Congratulations!

You've created a **professional Date Dimension** similar to what enterprise Power BI developers use.

Lesson 8 Preview – Relationships (The Heart of Data Modeling)

Currently, you have two independent tables:

Calendar

FactCalls

They don't know about each other yet.

In the next lesson, you'll learn:

- What relationships are.
- One-to-Many (1:*) relationships.
- Why Calendar[Date] should connect to FactCalls[Date].
- Active vs. Inactive relationships.
- Filter direction.
- Why star schema improves performance.

- How the relationship powers all your charts automatically.

This is the point where your data model becomes a true analytical model rather than just a collection of tables.

Excellent. Before we continue with **Lesson 7 (Part 2)**, let's first make sure your **Calendar[Date]** column is saved correctly as a **Date** (not Date/Time). This is an important prerequisite.

Step 1 – Select the Calendar Table

Go to **Table View** (the table icon on the left).

Click the **Calendar** table.

You should see:

Date
01-01-2024 00:00:00
02-01-2024 00:00:00
...

Step 2 – Select the Date Column

Click the **Date** column header.

The entire column should become selected.

Step 3 – Open Column Tools

At the top ribbon, you'll see **Column tools** (or **Table tools** depending on your Power BI version).

Click **Column tools**.

You should see options like:

Name

Data type

Format

Summarization

Sort by column

Step 4 – Change Data Type

Find:

Data type

If it shows:

Date/Time

change it to:

Date

Power BI may ask:

"Do you want to change the data type?"

Click:

Replace Current

Step 5 – Change the Display Format

Still in **Column tools**, find:

Format

Choose one of these:

Recommended:

dd-MMM-yyyy

Example:

01-Jan-2024

or

dd-MM-yyyy

Example:

01-01-2024

Both are fine.

Step 6 – Verify

Now your Calendar table should display:

Date
01-Jan-2024
02-Jan-2024
03-Jan-2024

instead of:

Date
01-Jan-2024 00:00:00

Why Do We Do This?

A **Date Dimension** should contain **dates only**.

Keeping it as a Date (instead of Date/Time):

- Makes relationships clearer.
- Avoids issues if time values appear later.
- Is the standard approach in Power BI data models.

Step 7 – Verify FactCalls Date

Now click the **FactCalls** table.

Select the **Date** column.

Check:

- **Data type:** Date
- **Format:** dd-MMM-yyyy (or your preferred date format)

The two columns should match:

Table	Column	Data Type
Calendar	Date	Date
FactCalls	Date	Date

This is important because we'll create a relationship between these columns in the next lesson.

Save Your Work

Press:

Ctrl + S

or click the **Save** icon.

This completes the preparation for the Calendar table.

Lesson 7 (Part 1) Completed

You have successfully:

-  Imported and cleaned the data.

-  Loaded the FactCalls table.
-  Created the Calendar table.
-  Converted Calendar[Date] to the **Date** data type.
-  Verified the date format.

Now your model is ready for **Lesson 7 (Part 2)**, where we'll enrich the Calendar table by adding:

- Year
- Quarter
- Month Number
- Month Name
- Day Number
- Day Name
- Week Number
- Is Weekend

These additions will turn your simple Calendar table into a professional Date Dimension used in enterprise Power BI solutions.

Excellent catch! I can see exactly what happened from your screenshot.

The problem is not your DAX formula.

The problem is that you created a **Measure** instead of a **Column**.

Why You're Getting This Error

Your formula is:

DAX

Year = YEAR(Calendar[Date])

This formula is **100% correct only for a calculated column**.

But the error says:

"A single value for column 'Date' in table 'Calendar' cannot be determined."

This error occurs because **measures** don't work row by row. A measure sees the **entire Date column**, not one date at a time.

I Can Confirm This From Your Screenshot

On the right side, I see:

Calendar

Date

FactCalls

...

Table

Year ⚠️

Notice **Year** appears as its own item, **not under the Calendar table**.

That means **Year is a Measure**, not a column.

Fix It

Step 1

Delete the incorrect measure.

On the right pane:

Right-click

Year

↓

Choose

Delete

Step 2

Click the **Calendar** table first.

Make sure **Calendar** is selected.

Step 3

Click

Table tools

↓

New column

⚠ **Do NOT click "New measure".**

Use **New column**.

Step 4

Now enter exactly this:

DAX

Year = YEAR(Calendar[Date])

Press **Enter**.

Now Power BI evaluates the formula **for each row**.

Example:

Date	Year
01-Jan-2024	2024
02-Jan-2024	2024
03-Jan-2024	2024

Perfect!

Why Does It Work as a Column?

A calculated column processes one row at a time.

Imagine this table:

Date
01-Jan-2024
02-Jan-2024
03-Jan-2024

Power BI internally does:

Row 1

YEAR(01-Jan-2024)

↓

2024

Row 2

YEAR(02-Jan-2024)

↓

2024

Row 3

YEAR(03-Jan-2024)

↓

2024

Every row gets its own value.

Why Doesn't It Work as a Measure?

A measure sees:

Calendar[Date]

which represents **all dates**.

Like:

01-Jan

02-Jan

03-Jan

04-Jan

...

31-Dec

Now if you ask:

DAX

YEAR(Calendar[Date])

Power BI asks:

"Which date?"

There are hundreds of them.

So it throws the error.

Quick Rule to Remember

Use **New Column** when:

- Year
- Month
- Quarter
- Day Name
- Category
- Flags (Weekend/Weekday)

These are stored for **every row**.

Use **New Measure** when:

- Total Calls
- Average Resolution Time
- Pending Calls
- Completed Calls
- SLA %
- KPIs

Measures calculate values **on demand** for visuals.

Your Next Steps

After creating the **Year** column correctly, create these columns (using **New column** each time):

DAX

Quarter = "Q" & QUARTER(Calendar[Date])

DAX

Month Number = MONTH(Calendar[Date])

DAX

Month Name = FORMAT(Calendar[Date], "MMMM")

DAX

Day Number = DAY(Calendar[Date])

DAX

Day Name = FORMAT(Calendar[Date], "dddd")

DAX

Week Number = WEEKNUM(Calendar[Date], 2)

DAX

Is Weekend =

IF(

WEEKDAY(Calendar[Date], 2) > 5,

"Yes",

"No"

)

Before You Continue

Please create the **Year** as a **New Column** and send me a screenshot. I'll verify it's correct before we build the remaining columns and then move on to creating relationships.

Yes, **testing the relationship in Report View is absolutely the right thing to do**. In real projects, developers always verify the model before building the dashboard.

However, in your screenshot, you're testing it with the **wrong fields**.

What I noticed

Your chart uses:

- **Y-axis:** Month Name
- **X-axis:** Sum of Year

This is why you're getting values around **50K**.

Power BI is **adding the Year values**:

For example:

2024 + 2024 + 2024 + 2024 + ...

So January becomes something like:

2024 × Number of January dates

≈ 62,744

This number has **no business meaning**.

How to Test the Relationship Correctly

Instead of summing the Year column, count the records in the **FactCalls** table.

Method 1 (Recommended)

First create this measure:

DAX

Total Calls =

COUNTROWS(FactCalls)

Then create a chart:

- **Axis:** Calendar[Month Name]
- **Values:** Total Calls

Expected result:

January [REDACTED]

February [REDACTED]

March [REDACTED]

April [REDACTED]

...

This proves that:

- The Calendar table is filtering the FactCalls table.
 - The relationship is working.
-

Method 2 (Without Creating a Measure)

Create a **Table** visual.

Add:

- Calendar[Date]
- FactCalls[Problem]

If the relationship is working:

Date	Problem
01-Jan-2024	Printer Issue
02-Jan-2024	LAN Issue
03-Jan-2024	(Blank if no calls)

Notice that dates with no calls still appear because they come from the Calendar table.

Method 3 (Best Test)

Create:

- A **Slicer** using Calendar[Month Name]

- A **Card** using:

DAX

Total Calls = COUNTROWS(FactCalls)

Now click:

January

The card should show only January calls.

Click:

April

The card should change to April calls.

If the number changes correctly, the relationship is working.

One More Important Fix

Your Month Name will currently sort alphabetically:

April

August

December

February

January

...

You already created **Month Number**.

Now fix the sorting:

1. Select **Calendar → Month Name**.
2. Go to **Column tools**.
3. Click **Sort by column**.

4. Choose **Month Number**.

Now the months will display correctly:

January

February

March

April

May

June

July

August

September

October

November

December

Professional Validation Checklist

Before moving to the dashboard, verify these items:

- ☒ Relationship: Calendar[Date] → FactCalls[Date]
- ☒ Cardinality: 1:*
- ☒ Cross-filter direction: Single
- ☒ Relationship is Active
- ☒ Month Name sorted by Month Number
- ☒ Calendar[Date] and FactCalls[Date] are both **Date** type
- ☒ COUNTROWS(FactCalls) changes when filtering by Calendar fields

Next Step

Once you've confirmed the relationship works, **stop testing with raw columns** like Year.

The next professional step is to create a **Measures** table and start building reusable DAX measures (Total Calls, Completed Calls, Pending Calls, etc.). Those measures will power every KPI card, chart, and dashboard page we build from here onward.

From your screenshot, I actually see **two things**:

1. **The relationship appears to be working.** ✓
2. **The month order is wrong.** ✗

Let's verify each one.

1. Are all bars equal because something is wrong?

Not necessarily.

If you generated **the same number of helpdesk calls for every month** (for example, 180 calls in each monthly Excel file), then this result is **expected**.

Example:

Month	Calls
January	180
February	180
March	180
April	180

The chart will show bars of equal length.

Quick test

Create a **Table** visual with:

- Calendar[Month Name]
- Total Calls

If it shows something like:

Month	Total Calls
January	180
February	180
March	180
...	...

then your relationship is working correctly.

2. The month order is wrong (This definitely needs fixing)

Your chart shows:

April

August

December

February

January

...

This is **alphabetical order**, not calendar order.

Fix it

1. Go to **Table View**.
2. Select **Calendar → Month Name**.

3. Click **Column tools**.
4. Click **Sort by column**.
5. Choose **Month Number**.

Now the chart will display:

January

February

March

April

May

June

July

August

September

October

November

December

3. Verify the Relationship

Let's make sure the relationship is really filtering.

Create a **Slicer**:

- Calendar[Month Name]

Create a **Card**:

- Total Calls

Now click **January** in the slicer.

- Does the card change?

Then click **February**.

- Does it change again?

If YES

✓ Your relationship is working.

If NO

Then we'll inspect the relationship and the date columns.

4. Verify the Date Columns

Check these two columns:

- Calendar[Date]
- FactCalls[Date]

Both should be:

- **Data Type:** Date
- **Not** Date/Time

Also make sure the relationship is:

Calendar[Date] (1)

|



FactCalls[Date] (*)

with:

- Cardinality: **One-to-Many**
 - Cross-filter direction: **Single**
 - Active: **Yes**
-

5. One More Important Question

Please tell me:

How many rows does each monthly Excel file contain?

For example:

- January.xlsx → 180 rows
- February.xlsx → 180 rows
- March.xlsx → 180 rows

or

- January → 150
- February → 220
- March → 175

If every file has exactly the same number of rows, then **equal bars are the correct result**, not a modeling problem.

Before We Move On

Please send me **one screenshot of your Model View** showing the relationship between Calendar and FactCalls.

That single screenshot will let me confirm:

- ☒ Relationship is correct
- ☒ Cardinality is correct
- ☒ Filter direction is correct
- ☒ No hidden modeling issue remains

Then we'll proceed to building the dashboard.

Excellent! You have completed the **Data Preparation** and **Data Modeling** phases. From now on, we'll build the dashboard exactly as it's done in a real company. I won't skip any steps, and I'll explain not just *what* to do, but *why* you're doing it.

Current Project Status

- ✓ Excel Files Imported
- ✓ Power Query Completed
- ✓ FactCalls Table Created
- ✓ Calendar Table Created
- ✓ Calendar Attributes Added
- ✓ Relationship Created
- ✓ Relationship Tested

Now we start the **Report Development** phase.

Phase 1 – Prepare the Semantic Model (Before Building Visuals)

Many beginners start creating charts now.

Professional developers don't.

They first prepare the model.

Step 1 – Create a Measures Table ★

Why?

Currently, if you create measures in FactCalls, after a few weeks you'll have:

FactCalls

Date

Problem

Department

Status

Solved By

...

Total Calls

Completed Calls

Pending Calls

Average Time

SLA

...

It becomes difficult to find columns and measures.

Instead, professionals create one dedicated table.

Create It

Go to:

Home

↓

Enter Data

Create:

Dummy
1

Table Name:

Measures

Click **Load**.

Hide Dummy Column

Right-click

Dummy

↓

Hide in report view

Now your model becomes

Calendar

FactCalls

Measures

This is the industry standard.

Step 2 – Create Measure Folders (Optional but Professional)

Later you'll have 40–60 measures.

We'll organize them into folders like:

Measures

KPIs

Time Intelligence

Technician

Department

SLA

For now, keep all measures in the Measures table.

Phase 2 – Create Core Measures

These measures are the foundation of the dashboard.

Create them one by one using:

Modeling

↓

New Measure

Measure 1

DAX

Total Calls =

COUNTROWS(FactCalls)

Purpose:

Counts every helpdesk ticket.

Measure 2

DAX

Completed Calls =

CALCULATE(

COUNTROWS(FactCalls),

FactCalls[Status]="Completed"

)

Purpose:

Shows completed tickets.

Measure 3

DAX

Pending Calls =

```
CALCULATE(  
    COUNTROWS(FactCalls,  
        FactCalls[Status]="Pending"  
    )
```

Purpose:

Shows pending tickets.

Measure 4

DAX

Average Resolution Time =

```
AVERAGE(FactCalls[Time Taken (Hours)])
```

Purpose:

Average hours taken to resolve tickets.

Measure 5

DAX

Total Departments =

```
DISTINCTCOUNT(FactCalls[Department])
```

Purpose:

Counts unique departments.

Measure 6

DAX

Total Technicians =

DISTINCTCOUNT(FactCalls[Solved By])

Purpose:

Counts technicians.

Step 3 – Validate Every Measure

Create a temporary table visual.

Add:

Total Calls

Completed Calls

Pending Calls

Average Resolution Time

Total Departments

Total Technicians

Verify:

- Numbers look reasonable.
- No errors.
- Filters change values correctly.

If everything works,

Save the project.

Ctrl + S

Phase 3 – Dashboard Planning

Before dragging visuals, decide the layout.

Professional dashboards are designed first.

Executive Dashboard

+-----+

| IT HELPDESK DASHBOARD |

+-----+

+-----+-----+-----+-----+-----+-----+

| Total | Completed | Pending | Avg Res | Depts | Techs |

| Calls | Calls | Calls | Time | | |

+-----+-----+-----+-----+-----+-----+

+-----+-----+

| Monthly Call Trend | Calls by Department |

+-----+-----+

+-----+-----+

| Problem Categories | Technician Performance |

+-----+-----+

+-----+

| Ticket Details |

+-----+

Notice:

- KPIs at the top.
- Trend charts in the middle.
- Detailed table at the bottom.

Phase 4 – Apply Report Theme

Before building visuals:

View

↓

Themes

Choose:

- Corporate
- Default
- Custom (later)

This keeps colors and fonts consistent.

Phase 5 – Create KPI Cards

We'll build:

- Total Calls
- Completed Calls
- Pending Calls
- Average Resolution Time
- Total Departments
- Total Technicians

Each KPI will use the measures you created.

Phase 6 – Create Charts

We'll build them in this order:

1. Monthly Call Trend
2. Department-wise Calls
3. Problem Category Analysis
4. Technician Performance
5. Resolution Time Analysis
6. Holiday vs Working Day Analysis

Phase 7 – Create Slicers

We'll add:

- Year
- Quarter
- Month
- Department
- Technician
- Problem
- Holiday
- Status

Every visual will respond to these slicers.

Phase 8 – Build Detail Page

We'll create a searchable table showing:

- Date
 - Department
 - User
 - Problem
 - Technician
 - Status
 - Resolution Time
-

Phase 9 – Navigation

We'll add:

- Buttons
 - Bookmarks
 - Drill-through
 - Tooltips
-

Phase 10 – Optimization

We'll learn:

- Model optimization
 - Performance Analyzer
 - Best practices
 - Publishing
 - Scheduled Refresh
-

Learning Roadmap

Module 1 Dataset Design 

Module 2 Power Query 

Module 3 Calendar Table 

Module 4 Relationships 

Module 5 Measures (Current)

|



Module 6 Dashboard Layout

|



Module 7 KPI Cards

|



Module 8 Charts

|



Module 9 Slicers

|



Module 10 Detail Reports

|



Module 11 Navigation

|



Module 12 Publish

What We'll Do Next (Lesson 9)

We'll **start with the Measures table** and build the six core measures. These measures are the backbone of the dashboard and will be reused in every KPI card, chart, and report page.

Once the measures are ready and tested, we'll move on to **Lesson 10**, where we'll design the dashboard layout and create the KPI cards one by one.

Excellent questions. These are exactly the questions a Power BI developer should ask. Let's address them one by one.

Question 1: Why does Total Calls show only 180?

From your screenshot:

- The **Card** shows **180**.
- The **chart** shows **180 for every month**.

This tells me something important:

Your **Total Calls** measure is correct, but your current dataset likely contains **only one month's data** or **exactly 180 rows per month**.

Let's verify.

Step 1: Check the FactCalls table

Go to **Table View**.

Select **FactCalls**.

Look at the bottom-left or bottom-right status bar.

How many rows does it show?

Examples:

FactCalls

Rows: 180

or

Rows: 2160

Tell me that number.

Step 2: Verify Source.Name

In **Table View**, check the Source.Name column.

Do you see:

2024_January.xlsx

2024_February.xlsx

2024_March.xlsx

...

or only

2024_April.xlsx

If you only see one file name, then only one file was imported.

Step 3: Check Power Query

Open **Transform Data**.

Click **FactCalls**.

Look at the Source.Name column.

Do you see multiple Excel file names?

If yes, all files are loaded.

If no, only one file is loaded.

Question 2: If I keep adding Excel files, won't the table become huge?

Yes, it will.

But that's exactly how a **Fact Table** is supposed to work.

Imagine this:

Year	Tickets
2024	2,160
2025	2,450
2026	2,800
2027	3,100

After four years:

10,510 rows

Power BI is designed for this.

Is 10,000 rows a problem?

No.

Power BI easily handles:

- 100,000 rows
- 1 million rows
- 10 million+ rows (depending on hardware and model design)

Your helpdesk dataset is small by Power BI standards.

What happens when a new month arrives?

Suppose next month you receive:

2026_August.xlsx

You copy it into:

Data\

Then click:

Refresh

Power Query automatically:

January

February

March

...

July

August

combines them into one FactCalls table.

You don't manually merge anything.

Why One Big Fact Table?

Imagine keeping separate tables:

January

February

March

April

...

Now imagine answering:

"Show total calls for 2024."

Power BI would have to query 12 different tables.

Instead, with one Fact table:

FactCalls

Date

Department

Problem

...

it's just one query.

This is much faster and much easier to maintain.

How Power BI Handles Large Tables

Power BI doesn't read the Excel files every time you click a chart.

Instead:

Excel Files

|



Power Query

|



Compressed In-Memory Model (VertiPaq)

|



Dashboard

The VertiPaq engine compresses data heavily.

Example:

An Excel folder of **50 MB** may occupy only **8–15 MB** in memory, depending on the data.

Will Performance Become Slow?

Not if you follow best practices:

- Use a **Star Schema** (which you're doing).
 - Remove unused columns.
 - Use measures instead of calculated columns where appropriate.
 - Set correct data types.
 - Avoid unnecessary relationships.
 - Keep the model clean.
-

Your Current Project Size

If each month has:

180 rows

Then:

$12 \times 180 = 2,160$ rows

Even if you keep five years:

$2,160 \times 5 = 10,800$ rows

Power BI treats this as a very small model.

One Thing I Notice in Your Screenshot

Your **Total Calls** measure is under the **Calendar** table.

That's not the recommended approach.

Professionally, measures are kept in a separate **Measures** table.

Instead of:

Calendar

Total Calls

you should eventually have:

Measures

Total Calls

Completed Calls

Pending Calls

Average Resolution Time

We'll reorganize them in the next lesson.

Next Step

Before we continue, please check **one thing**:

Go to **Table View** → **FactCalls** and tell me:

How many rows does the FactCalls table contain?

Also tell me whether the Source.Name column contains:

- only one file name, or
- all 12 monthly Excel file names.

That will immediately tell us whether you're seeing **180** because only one file is loaded or because each month intentionally contains **180 calls**.

Perfect! This tells us something very important.

Your FactCalls table has **2,166 rows**

That means:

-  All 12 Excel files have been combined successfully.
-  Your Folder import is working.
-  Power Query is working.
-  Your Relationship is probably working.

So why does your card show **180** instead of **2,166**?

Most Likely Cause

From your screenshot, I noticed something important:

Your **Total Calls** measure is inside the **Calendar** table.

I suspect you created it like this:

DAX

Total Calls = COUNTROWS(Calendar)

instead of

DAX

Total Calls = COUNTROWS(FactCalls)

Check Your Measure

Click on **Total Calls**.

Look at the formula bar.

It should be exactly:

DAX

Total Calls =

COUNTROWS(FactCalls)

If it says:

DAX

COUNTROWS(Calendar)

then 180 is correct because your Calendar table has about 180 dates.

Why 180?

Your Calendar probably covers:

01-Jan-2024

to

29-Jun-2024

which is around:

180 days

So:

DAX

COUNTROWS(Calendar)

returns:

180

Correct Formula

Create or edit the measure:

DAX

Total Calls =

COUNTROWS(FactCalls)

Now the card should show approximately:

2166

Verify in 30 Seconds

Select the measure.

Tell me **exactly** what formula you see in the formula bar.

For example:

DAX

Total Calls = COUNTROWS(FactCalls)

or

DAX

Total Calls = COUNTROWS(Calendar)

That will immediately confirm the issue.

Another Small Improvement

Create a **Measures** table now.

Currently you have:

Calendar

Total Calls

Professionally it should be:

Measures

Total Calls

Completed Calls

Pending Calls

We'll move all measures there later.

After We Fix This

We'll continue with the real dashboard development:

1. ☒ Create all KPI Measures
2. ☒ Build Executive Dashboard
3. ☒ Create Monthly Trend
4. ☒ Department Analysis
5. ☒ Technician Performance
6. ☒ Problem Analysis
7. ☒ Interactive Slicers
8. ☒ Drill-through Pages
9. ☒ Publish & Automatic Refresh

You're now entering the most interesting part of the project—turning the data model into a professional dashboard.

Excellent. From now on, we'll stop thinking like a Power BI learner and start thinking like a **Power BI Dashboard Designer**.

Most beginners do this:

Import Data

↓

Create Relationship

↓

Start Dragging Charts Randomly ❌

Professional developers do this:

Import Data



Power Query



Data Modeling



Measures



Dashboard Planning ✅



Dashboard Design

↓

Visual Development

↓

Testing

↓

Optimization

↓

Publishing

The **Dashboard Planning** phase is one of the most important phases because a poorly planned dashboard confuses users even if the data is correct.

Phase 3 – Dashboard Planning

What is Dashboard Planning?

Dashboard planning is deciding:

- What information the user needs first.
- Which KPIs should be at the top.
- Which charts should be in the middle.
- Which details should be at the bottom.
- Which filters should always be visible.
- How the user will navigate the report.

Think of it like designing a house.

You don't start by placing furniture—you first create the floor plan.

Step 1 – Understand the Business Requirement

Before opening Power BI, ask:

Who will use this dashboard?

In your case:

- IT Manager
- Helpdesk Manager
- System Administrator
- Technician
- Department Heads

Each has different needs.

IT Manager wants

- Total tickets

- Pending tickets
- SLA performance
- Technician workload
- Monthly trends

Technician wants

- Assigned tickets
- Average resolution time
- Open tickets

Department Head wants

- Tickets from their department
- Frequent issues
- Resolution speed

A single dashboard cannot satisfy everyone, so we build multiple pages.

Step 2 – Identify the KPIs

KPIs are the numbers users look at first.

For your project, the top KPIs should be:

KPI	Why
Total Calls	Overall workload
Completed Calls	Work completed
Pending Calls	Outstanding work
Average Resolution Time	Efficiency
Total Departments	Coverage
Total Technicians	Resources
SLA % (later)	Service quality

These will become **Card visuals**.

Step 3 – Decide the Dashboard Layout

A good dashboard follows the user's eye movement.

Recommended Layout

+-----+

| IT HELPDESK DASHBOARD |

+-----+

+-----+-----+-----+-----+-----+-----+

| Total | Completed | Pending | Avg Time | Depts | Techs |

+-----+-----+-----+-----+-----+-----+

+-----+-----+

| Monthly Trend | Problem Categories |

+-----+-----+

+-----+-----+

| Department Analysis | Technician Performance |

+-----+-----+

+-----+

| Ticket Details Table |

+-----+

Notice:

- KPIs at the top.
 - Summary charts in the middle.
 - Detailed information at the bottom.
-

Step 4 – Decide the Dashboard Size

Go to:

Format

↓

Canvas Settings

Choose:

16:9

This fits most laptop and desktop screens.

Avoid very long pages that require excessive scrolling.

Step 5 – Choose a Theme

Go to:

View

↓

Themes

Choose one theme and keep it throughout the report.

Professional reports use consistent:

- Colors
- Fonts
- Borders
- Shadows

Example:

Blue → Information

Green → Success

Orange → Warning

Red → Critical

Step 6 – Decide the Color Meaning

Colors should have meaning.

Example:

Metric	Color
Completed	Green
Pending	Orange
Critical	Red
Information	Blue

Don't color visuals randomly.

Step 7 – Decide Visual Types

Choose the best visual for each question.

Business Question	Best Visual
Total Calls	Card
Monthly Trend	Line Chart
Calls by Department	Bar Chart
Calls by Technician	Column Chart
Problem Categories	Donut or Bar Chart
Ticket Details	Table

Choosing the wrong visual makes the report harder to understand.

Step 8 – Plan the Filters

Users shouldn't scroll through data manually.

Plan slicers for:

Year

Quarter

Month

Department

Technician

Problem

Holiday

Status

Place slicers together—typically on the left or across the top.

Step 9 – Design for Interaction

Think about what should happen when users click a chart.

Example:

User clicks:

Department 5

Automatically update:

- KPI Cards
- Monthly Trend
- Problem Analysis
- Technician Analysis

All visuals should interact through your relationships and measures.

Step 10 – Plan Multiple Report Pages

Don't overload one page.

Page 1 – Executive Summary

Shows overall performance.

Page 2 – Department Analysis

Focus on departments.

Page 3 – Technician Performance

Focus on technicians.

Page 4 – Problem Analysis

Analyze issue types.

Page 5 – Ticket Details

Provide searchable ticket-level data.

Step 11 – Sketch Before Building

A simple paper sketch is often enough.

Example:

IT HELPDESK DASHBOARD

Total Completed Pending Avg Time

Monthly Trend Problem Categories

Department Technician

Ticket Details

This avoids rearranging visuals later.

Step 12 – Save Your Work

After planning:

Ctrl + S

Planning doesn't change the data, but it saves your report layout and model.

Phase 3 Deliverables

By the end of this phase, you should have:

- A clear dashboard layout.
- A list of KPIs.
- A list of charts.
- A list of slicers.
- A page structure.
- A chosen theme.
- A plan for user interaction.

Only after this should you start placing visuals.

What Comes Next?

The remaining phases are:

Phase 4 – Dashboard Design

You'll create the visual layout based on the plan:

- Set page size.
- Apply the theme.
- Add a title.
- Create sections for KPIs and charts.
- Align and size placeholders.

Phase 5 – KPI Cards

Create Card visuals for:

- Total Calls
- Completed Calls
- Pending Calls
- Average Resolution Time

- Total Departments
- Total Technicians

You'll learn formatting, conditional colors, tooltips, and dynamic titles.

Phase 6 – Charts

Build and configure:

- Monthly Call Trend
- Department-wise Calls
- Problem Category Analysis
- Technician Performance
- Resolution Time Analysis
- Holiday vs Working Day Analysis

You'll learn how to choose the right chart and configure axes, legends, labels, and sorting.

Phase 7 – Tables and Matrices

Create detailed ticket views with:

- Conditional formatting
- Icons
- Search
- Export-ready layouts

Phase 8 – Slicers and Filters

Add interactive slicers for:

- Year
- Quarter
- Month
- Department

- Technician
- Problem
- Holiday
- Status

Then configure visual interactions and synchronized slicers across pages.

Phase 9 – Navigation

Enhance usability with:

- Buttons
- Bookmarks
- Drill-through pages
- Tooltip pages

Phase 10 – Optimization

Optimize the report by:

- Removing unused columns
- Optimizing DAX
- Checking performance with Performance Analyzer
- Reducing model size

Phase 11 – Publish and Refresh

Publish to the Power BI Service, configure scheduled refresh, and ensure that dropping a new monthly Excel file into the folder followed by a refresh updates the entire dashboard automatically.

Our next lesson will be Phase 4 – Dashboard Design, where we'll create the report canvas, apply a professional theme, and build the dashboard layout before placing any KPI cards or charts. This foundation makes the rest of the report much easier to build and maintain.

Excellent. You have now completed:

-  Data Collection
-  Power Query (ETL)
-  Data Modeling
-  Relationships
-  Dashboard Planning

Now we enter the phase where your Power BI report starts looking like a professional dashboard.

Phase 4 – Dashboard Design

Goal

By the end of this phase, you will have:

- A professional dashboard canvas
- A consistent color theme
- Proper spacing and alignment
- KPI placeholders
- Chart placeholders
- Slicer area
- Navigation-ready layout

Important: In this phase, we focus on **design and layout only**. We'll add the actual visuals in the next phase.

Step 1 – Create Report Pages

Don't keep everything on one page.

Rename **Page1** to:

Executive Dashboard

Then create these pages:

Executive Dashboard

Department Analysis

Technician Performance

Problem Analysis

Ticket Details

Why?

Each page has a specific purpose:

Page	Purpose
Executive Dashboard	Overall summary
Department Analysis	Department-wise insights
Technician Performance	Technician KPIs
Problem Analysis	Issue trends
Ticket Details	Searchable ticket list

Step 2 – Set Canvas Size

Click on a blank area of the report (not on a visual).

Open the **Format** pane.

Navigate to:

Canvas settings

↓

Type

Choose:

16:9

Why 16:9?

- Fits laptops
 - Fits projectors
 - Fits Power BI Service
 - Industry standard
-

Step 3 – Turn on Gridlines

Go to:

View

Enable:

- ☒ Gridlines
- ☒ Snap to Grid

Why?

They help align visuals precisely.

Without them, dashboards often look uneven.

Step 4 – Choose a Theme

Go to:

View



Themes

Choose a clean theme.

For an IT dashboard, blue and gray work well.

Example color palette:

Item	Color
Background	White
KPI Cards	Light Gray
Primary Accent	Blue
Completed	Green
Pending	Orange
Critical	Red

Step 5 – Set the Background

Click a blank area.

Go to:

Format



Canvas Background

Set:

- Color: White or very light gray
- Transparency: 0%

Avoid dark backgrounds for business reports unless your organization specifically uses them.

Step 6 – Add the Dashboard Title

Go to:

Insert

↓

Text Box

Type:

IT HELPDESK DASHBOARD

Format:

- Font: Segoe UI
- Size: 24–30
- Bold
- Centered

Below it, add a subtitle:

IT Support Performance Dashboard

Smaller and lighter than the main title.

Step 7 – Reserve Space for KPI Cards

At the top of the page, reserve six equal areas.

Example layout:

+-----+

| IT HELPDESK DASHBOARD |

+-----+

+-----+-----+-----+-----+-----+-----+

| Total | Completed | Pending | Avg Res | Depts | Techs |
| Calls | Calls | Calls | Time | | |

+-----+-----+-----+-----+-----+-----+

Don't worry about the values yet.

We're planning the layout.

Step 8 – Reserve Space for Charts

Below the KPI row:

+-----+

| Monthly Call Trend | Problem Categories |

+-----+

Next row:

+-----+

| Department Analysis | Technician Performance |

+-----+

Bottom:

+-----+

| Ticket Details Table |

+-----+

This creates a logical reading order.

Step 9 – Reserve Space for Slicers

Decide where slicers will go.

Option 1 (Recommended)

Left panel:

Year

Month

Department

Technician

Problem

Holiday

Status

Option 2

Top horizontal row.

For your project, I recommend the **left panel** because you have several filters.

Step 10 – Align Everything

Use:

Format

↓

Align

Commands you'll use frequently:

- Align Left
- Align Right
- Align Top
- Align Bottom
- Distribute Horizontally
- Distribute Vertically

Consistent spacing makes the dashboard look polished.

Step 11 – Save

Press:

Ctrl + S

This is your first design checkpoint.

Phase 4 Deliverables

By the end of this phase, your Executive Dashboard should have:

- No charts or KPI values yet—that comes next.

Phase 5 – KPI Cards

We'll create these cards:

You'll learn:

- How to use **Card** visuals
- Card formatting
- Conditional colors
- Icons

- Dynamic titles
- Number formatting
- Tooltips
- Alignment and spacing

After that, we'll build the charts, slicers, and interactions one step at a time.

Absolutely. From this point onward, I'll teach you **like a professional Power BI trainer**. I won't skip any clicks or assume prior knowledge. I'll explain:

- **Where to click**
 - **Why you're clicking it**
 - **What happens internally**
 - **What you should expect to see**
 - **Common mistakes**
 - **Professional best practices**
-

Phase 4 – Dashboard Design (Hands-on)

Lesson 1 – Create a Professional Dashboard Canvas

Objective: Before adding any charts or cards, prepare a clean, professional report page.

Step 1 – Rename the Report Page

Why?

Instead of generic names like **Page 1**, use meaningful names. This makes navigation easier as your report grows.

Instructions

1. Look at the bottom-left corner of Power BI.
2. Find the tab named:

Page 1

3. Right-click **Page 1**.
4. Click:

Rename

5. Type:

Executive Dashboard

6. Press **Enter**.

Result

You should now see:

Executive Dashboard

instead of

Page 1

Step 2 – Create All Report Pages

Why?

Professional reports separate different analyses instead of putting everything on one page.

Instructions

1. At the bottom, click the + button.
2. A new page appears.
3. Right-click the new page.
4. Rename it:

Department Analysis

Repeat to create:

Technician Performance

Problem Analysis

Ticket Details

Expected Result

Your page tabs should look like:

Executive Dashboard

Department Analysis

Technician Performance

Problem Analysis

Ticket Details

Step 3 – Save Your Project

Why?

Always save after completing a milestone.

Instructions

Click:

File

↓

Save

or simply press:

Ctrl + S

Step 4 – Set Canvas Size

Why?

The canvas is the area where you build your dashboard.

Using the correct size ensures the report looks good on laptops, desktops, and Power BI Service.

Instructions

1. Click on an **empty white area** of the report.
 - o Don't click on a visual.
2. On the right, open the **Format** pane (paint roller icon if needed).
3. Expand:

Canvas settings

4. Find:

Type

5. Select:

16:9

Why 16:9?

It's the standard widescreen ratio and works well in most business environments.

Step 5 – Turn On Gridlines

Why?

Gridlines help you align visuals neatly.

Instructions

1. Click:

View

2. Enable:

✓ Gridlines

3. Enable:

✓ Snap to Grid

What Happens?

You'll see faint lines across the canvas.

When you move visuals, they snap into alignment.

Step 6 – Choose a Theme

Why?

A theme applies consistent colors and fonts throughout your report.

Instructions

1. Click:

View

2. Click:

Themes

3. Choose one theme (for example, the default Office theme).

You can customize colors later.

Step 7 – Set the Background

Why?

A clean background improves readability.

Instructions

1. Click an empty area of the canvas.
2. Open the **Format** pane.
3. Expand:

Canvas background

Set:

Property	Value
Color	White or Light Gray
Transparency	0%

Step 8 – Add the Dashboard Title

Why?

Every report should clearly identify its purpose.

Instructions

1. Click:

Insert

2. Click:

Text Box

3. Type:

IT HELPDESK DASHBOARD

Format it:

Property	Value
Font	Segoe UI
Size	26–30
Bold	Yes
Alignment	Center

Below it, add another text box:

IT Support Performance Dashboard

Use a smaller font (around 12–14 pt).

Step 9 – Plan the KPI Area

Don't add cards yet.

Instead, visualize where they'll go.

Imagine this row:

```
+-----+-----+-----+-----+-----+-----+
| Total | Completed | Pending | Avg Res | Depts | Techs |
| Calls | Calls | Calls | Time | | |
+-----+-----+-----+-----+-----+-----+
```

Leave enough space for six equally sized cards.

Step 10 – Plan the Chart Area

Reserve two rows beneath the KPIs.

First row:

+-----+-----+
| Monthly Call Trend | Problem Categories |
+-----+-----+

Second row:

+-----+-----+
| Department Analysis | Technician Performance |
+-----+-----+

Step 11 – Reserve Space for Ticket Details

At the bottom, leave a wide area for a table.

+-----+
| Ticket Details |
+-----+

Step 12 – Plan the Slicer Area

I recommend placing slicers on the **left**.

Reserve a vertical strip for:

Year

Quarter

Month

Department

Technician

Problem

Holiday

Status

This keeps filters together and leaves more room for charts.

Step 13 – Align the Layout

As you begin adding visuals later, use:

Format

↓

Align

Commands such as:

- Align Left
- Align Top
- Distribute Horizontally
- Distribute Vertically

These keep spacing consistent.

Step 14 – Save Again

Press:

Ctrl + S

This marks the end of the dashboard layout phase.

What You Should Have Now

Your report should contain:

- ☒ Five report pages
- ☒ Executive Dashboard selected
- ☒ Canvas size set to 16:9
- ☒ Gridlines enabled
- ☒ Snap to Grid enabled
- ☒ Theme selected
- ☒ White (or light gray) background
- ☒ Dashboard title

-  Planned areas for KPIs, charts, slicers, and detail table
-

Before We Continue

We're **not** adding charts yet.

The next lesson is:

Phase 5 – KPI Cards (Practical)

We'll build the first six KPI cards step by step:

1. Total Calls
2. Completed Calls
3. Pending Calls
4. Average Resolution Time
5. Total Departments
6. Total Technicians

You'll learn:

- Which visual to choose
- Which measure to drag
- How to format fonts, colors, labels, and backgrounds
- How to align all cards perfectly
- How to create a polished executive summary row

From this point onward, every lesson will be similarly hands-on, with exact Power BI actions and explanations.

Excellent! Now we start the phase where your dashboard comes alive.

You have completed:

-  Data Import

-  Power Query
-  Calendar Table
-  Relationships
-  Dashboard Planning
-  Dashboard Layout

Now we'll build the **Executive Dashboard** exactly like a professional Power BI developer.

Phase 5 – KPI Cards (Executive Dashboard)

Lesson 1 – Create the First KPI Card (Total Calls)

What is a KPI Card?

A **KPI (Key Performance Indicator)** card displays one important business metric.

Examples:

Total Calls
2,166

A manager should understand the dashboard in **5–10 seconds** by looking at these cards.

KPI Cards We Will Create

We'll create them one by one.

| Total |

| Calls |

| Completed |

| Calls |

| Pending |

| Calls |

| Avg Time |

| Total |
| Departments |
|

|
| Total |
| Technicians |
|

Lesson 5.1 – Total Calls Card

Step 1 – Go to Executive Dashboard

At the bottom

Click

Executive Dashboard

Step 2 – Open Visualizations

On the right side you'll see

Data

Visualizations

Format

If Visualizations isn't visible,

click

View



Panes



Visualizations

Step 3 – Select Card Visual

Inside Visualizations

Find

Card

It looks like

123

or simply **Card** (depending on your Power BI version).

Click it once.

What Happens?

A blank card appears.

(Blank)

Don't resize it yet.

Step 4 – Add Measure

Expand

Measures

or wherever you stored your measures.

Find

Total Calls

Drag

Total Calls

↓

onto the card.

Expected Result

The card now shows

2166

or whatever your total row count is.

If it still shows **180**, check that the measure is:

DAX

Total Calls =

COUNTROWS(FactCalls)

Step 5 – Resize the Card

Select the card.

Drag the corners.

Recommended size

Width

≈ 180 px

Height

≈ 100 px

Don't worry about exact pixels.

Make it wide enough to read comfortably.

Step 6 – Position the Card

Move it to the

top-left

Total Calls

Leave some margin from the page edges.

Step 7 – Format the Card

Select the card.

Go to

Format

(Paint roller icon)

We'll configure it section by section.

Callout Value

Expand

Callout Value

Set:

Property	Value
Font Size	28–36
Bold	Yes
Display Units	None
Decimal Places	0

Category Label

Expand

Category Label

Turn it

ON

It should display

Total Calls

Set

Property	Value
Font Size	12–14
Bold	Yes

Background

Expand

Effects

↓

Background

Choose

White

or

Very Light Gray

Border

Turn

Border

ON

Radius

10

or

12

Makes it modern.

Shadow

Turn

Shadow

ON

Very small.

This gives a professional look.

Step 8 – Test the Card

Create a slicer.

Add

Calendar

↓

Month Name

Click

January

The card should change.

Click

February

The card should update.

This proves:

- Relationship works
 - Measure works
 - Filtering works
-

Step 9 – Duplicate the Card

Don't build every card from scratch.

Click the card.

Press

Ctrl + C

Ctrl + V

Now you have two identical cards.

Move the second one next to the first.

Repeat until you have six cards.

Card 1	Card 2	Card 3	Card 4	Card 5	Card 6

This keeps all sizes identical.

Step 10 – Replace Measures

Now select each copied card and replace the measure.

Card	Measure
Card 1	Total Calls
Card 2	Completed Calls
Card 3	Pending Calls
Card 4	Average Resolution Time
Card 5	Total Departments
Card 6	Total Technicians

Step 11 – Align the Cards

Select all six cards.

Go to

Format

↓

Align

↓

Align Top

Then

Format

↓

Distribute Horizontally

This creates even spacing.

Step 12 – Save

Press

Ctrl + S

What You Have Achieved

You now have the executive KPI row that appears at the top of almost every business dashboard.

+-----+

Total Calls | Completed | Pending | Avg Time | Departments | Techs

+-----+

These cards are interactive.

When the user changes a slicer (for example, selects a month or department), every card updates automatically because they're driven by your DAX measures and the relationship you created earlier.

Next Lesson – Phase 6

We'll build the **Monthly Call Trend** chart.

You'll learn:

- Why a **Line Chart** is better than a Column Chart for trends.
- Which fields go on the X-axis and Y-axis.
- How to sort months correctly.
- How to display months with **zero calls**.
- How to format axes, titles, gridlines, and data labels.
- How to make the chart fully interactive with the KPI cards and slicers.

This will be your first analytical visual, built step by step.

Excellent! 🎉

You now have:

- ☒ Data imported
- ☒ Power Query completed
- ☒ Calendar table created
- ☒ Relationship created

- ☒ Measures created
- ☒ KPI Cards created

Now we move to the **first analytical visualization**.

This is one of the most important charts in every business dashboard.

Phase 6 – Charts & Visualizations

Lesson 1 – Monthly Call Trend (Line Chart)

Business Requirement

Suppose your IT Manager asks:

"How are helpdesk calls changing over time?"

He doesn't want to see a table like this:

Month	Calls
Jan	180
Feb	175
Mar	192

He wants to identify trends visually.

A line chart is ideal because it highlights increases, decreases, and seasonal patterns.

Why Use a Line Chart?

A line chart is best when the X-axis is **time**.

Good examples:

- Daily calls
- Monthly calls
- Quarterly calls
- Yearly calls

Avoid using a pie chart or donut chart for trends.

Step 1 – Create Space for the Chart

On the **Executive Dashboard** page:

Below the KPI cards,

leave an area like this:

KPI Cards

Monthly Call Trend

+-----+

| |

| |

| |

+-----+

Step 2 – Insert a Line Chart

Instructions

1. Click on an empty area of the report.
2. In **Visualizations**, click:

Line Chart

(Icon showing a line with points.)

A blank chart appears.

Step 3 – Resize the Chart

Recommended size:

- Width: about half the page.
- Height: around 250–300 pixels.

Leave space on the right for another chart.

Step 4 – Position the Chart

Move it below the KPI cards.

Example:

KPI Cards

+-----+-----+

| Monthly Trend | Problem Categories |

| | |

+-----+-----+

Step 5 – Add Fields

This is the most important part.

X-Axis

Drag:

Calendar

↓

Month Name

to **X-axis**.

Y-Axis

Drag your measure:

Total Calls

to **Y-axis**.

The chart now displays monthly call counts.

Step 6 – Fix Month Sorting

If the months appear like:

April

August

December

February

January

they are sorted alphabetically.

Fix it:

1. Go to **Table View**.

2. Select:

Calendar

↓

Month Name

3. Click:

Column tools

↓

Sort by Column

4. Choose:

Month Number

Return to the report.

Now the months display correctly:

January

February

March

April

...

December

Step 7 – Show Months with Zero Calls

This is a professional feature.

Suppose there were no tickets in June.

We still want June to appear.

Instructions

Click the drop-down beside:

Month Name

inside the X-axis field well.

Enable:

Show items with no data

Because the chart uses the **Calendar** table, months without calls can still appear.

Step 8 – Format the Chart

Select the chart.

Open the **Format** pane.

General → Title

Turn **Title** ON.

Text:

Monthly Call Trend

Font:

- Segoe UI
- 16–18 pt
- Bold

X-axis

Set:

- Title: Off
 - Labels: On
 - Text size: 11–12
-

Y-axis

Title:

Total Calls

Display Units:

None

Decimal Places:

0

Data Labels

Turn **On**.

Now every point displays its value.

Example:

Jan ●180

Feb ●175

Mar ●192

Gridlines

Keep **major gridlines** on.

They make it easier to compare values.

Legend

Turn **Off**.

You only have one measure, so a legend isn't needed.

Line Style

Choose:

- Stroke Width: 2–3
- Markers: On

Markers make individual months easier to identify.

Step 9 – Test the Chart

Create a slicer using:

Department

Select:

HR

The line chart should update.

Then select:

Finance

The trend changes automatically.

This confirms:

- Relationship works.
 - Measure works.
 - Slicers work.
-

Step 10 – Copy the Chart

Instead of creating every chart from scratch:

Press:

Ctrl + C

Ctrl + V

Move the copy to the right.

Later, we'll change its fields to create another chart.

Common Mistakes

✗ Using FactCalls[Month Name] (if you had one).

Always use:

Calendar[Month Name]

because the Calendar table supports complete time analysis.

✗ Summing the Year column.

Always use a **measure** like:

DAX

Total Calls

✗ Forgetting to sort Month Name by Month Number.

Professional Checklist

Your chart should have:

- ☒ Line Chart
 - ☒ X-axis = Calendar[Month Name]
 - ☒ Y-axis = Total Calls
 - ☒ Month Name sorted by Month Number
 - ☒ Title: "Monthly Call Trend"
 - ☒ Data Labels ON
 - ☒ Markers ON
 - ☒ Legend OFF
 - ☒ Major Gridlines ON
-

End of Lesson 1

Your Executive Dashboard should now look like:

+-----+

| IT HELPDESK DASHBOARD |

+-----+

+-----+-----+-----+-----+-----+

| KPIs (6 Cards) |

+-----+-----+-----+-----+-----+-----+

+-----+-----+-----+-----+-----+-----+

| Monthly Call Trend | |

| | (Next Chart Placeholder) |

+-----+-----+-----+-----+-----+-----+

Next Lesson – Problem Category Analysis

We'll build a **horizontal bar chart** showing:

- Printer Issues
- Network Issues
- VPN Issues
- Windows Issues
- Email Issues
- Hardware Issues

You'll learn:

- Why a **bar chart** is better than a pie chart for many categories.
- How to sort by Total Calls.
- How to display Top N problems.
- Conditional colors.
- Dynamic tooltips.
- Drill-down capability.

By the end of that lesson, your dashboard will already look like a professional IT Helpdesk reporting solution.

Excellent. Your dashboard currently has:

KPI Cards (6)

Monthly Call Trend | (Empty Space)

(Empty Space) | (Empty Space)

Ticket Details

Now we'll build the **next two charts** that belong in the second row of your Executive Dashboard.

These are among the most commonly requested visuals by IT managers.

Monthly Call Trend | Problem Category Analysis

Department Analysis | Technician Performance

We'll build them one by one.

Chart 2 – Department-wise Calls

Business Question

Which departments generate the most IT helpdesk calls?

This helps identify departments that require more support or training.

Why a Horizontal Bar Chart?

Suppose you have 20 departments.

A vertical chart would make department names difficult to read.

A **horizontal bar chart** is much easier.

Step 1 – Insert the Visual

1. Go to **Executive Dashboard**.
2. Click an empty area below **Monthly Call Trend**.
3. In **Visualizations**, click:

Clustered Bar Chart

(Not Clustered Column Chart.)

A blank chart appears.

Step 2 – Resize

Place it in the **bottom-left**.

Example:

Department Analysis

Step 3 – Add Fields

Y-Axis (Categories)

Drag

FactCalls



Department

to **Y-axis**.

X-Axis (Values)

Drag

Measures



Total Calls

to **X-axis (Values)**.

Power BI automatically counts the calls per department.

Step 4 – Sort

Click the three dots (...) on the chart.

Choose:

Sort by



Total Calls



Descending

The department with the highest number of tickets appears at the top.

Step 5 – Format

Title

Turn ON.

Title:

Department-wise Calls

Data Labels

Turn ON.

Legend

Turn OFF.

Only one measure is displayed.

Gridlines

Keep ON.

Bars

Leave the default color for now.

Later, we'll apply conditional colors.

Final Result

Department-wise Calls

Finance 

HR 

Admin 

IT 

...

Chart 3 – Technician Performance

Business Question

Which technician resolved the most tickets?

Managers use this to evaluate workload distribution.

Why a Column Chart?

You only have six technicians.

A column chart makes comparing them easy.

Step 1 – Insert Visual

Click the empty area beside **Department Analysis**.

Choose

Clustered Column Chart

Step 2 – Resize

Place it in the **bottom-right**.

Technician Performance

Step 3 – Add Fields

X-Axis

Drag

FactCalls

↓

Solved By

Y-Axis

Drag

Measures

↓

Completed Calls

Step 4 – Sort

Sort

Completed Calls

↓

Descending

Step 5 – Format

Title

Technician Performance

Data Labels

ON.

Legend

OFF.

Gridlines

ON.

X-Axis

Rotate labels if technician names overlap.

Final Result

Technician Performance

Srinu 

Bhanu 

Mahesh 

Hameed 

Prabhakar 

Gyaneshwar 

Executive Dashboard Layout

After these two charts, your dashboard should look like this:

+-----+

| IT HELPDESK DASHBOARD |

+-----+

+-----+-----+-----+-----+-----+-----+

| Total | Completed | Pending | Avg Res | Depts | Techs |

| Calls | Calls | Calls | Time | | |

+-----+-----+-----+-----+-----+-----+

+-----+-----+

| Monthly Call Trend | Problem Category Analysis |

+-----+-----+

+-----+-----+

| Department-wise Calls | Technician Performance |

+-----+-----+

+-----+

| Ticket Details Table |

+-----+

Why This Layout?

Position	Reason
Top	KPIs provide an immediate summary.
First row	Trend and problem distribution show overall patterns.
Second row	Department and technician charts explain where the work is coming from and who is handling it.
Bottom	Detailed ticket table for investigation.

Professional Tips

- Use the **Format** → **Align** options after placing both charts.
 - Make both charts the **same height and width**.
 - Leave **10–15 pixels** of spacing between visuals.
 - Keep chart titles consistent in size and style.
-

Next Lesson

We'll build the **Problem Category Analysis** chart (top-right), including:

- Top N problem types
- Percentage of total calls
- Dynamic filtering with slicers
- Drill-down by problem category
- Professional formatting for business dashboards

That will complete all four major analytical visuals on your Executive Dashboard.

Excellent! 🎉

At this stage, your project should have:

- ☒ Data imported from the folder
- ☒ Power Query transformations completed
- ☒ Calendar table created
- ☒ Relationships created
- ☒ DAX measures created
- ☒ KPI Cards created
- ☒ Monthly Trend chart
- ☒ Department-wise Calls chart
- ☒ Technician Performance chart

Now we'll move to the next major phase.

Phase 7 – Advanced Dashboard Visualizations

This phase transforms your dashboard from a basic report into a management dashboard.

Dashboard Layout

By the end of this phase, your Executive Dashboard will look like this:

```
+-----+
| IT HELPDESK EXECUTIVE DASHBOARD |
+-----+
```


Lesson 1 – Problem Category Analysis

Business Question

Which problems are occurring most frequently?

Example:

Problem	Calls
Printer Issue	350
LAN Issue	320
VPN Issue	280
Windows Issue	240
Email Issue	200

Managers use this to identify recurring issues and plan preventive actions.

Why Use a Horizontal Bar Chart?

You have many problem categories with long names.

Horizontal bars display long labels more clearly than columns.

Step 1 – Insert Visual

1. Go to **Executive Dashboard**.
2. Click the empty area in the **top-right**.
3. In **Visualizations**, choose:

Clustered Bar Chart

Step 2 – Add Fields

Y-Axis

Drag:

FactCalls



Problem

X-Axis (Values)

Drag:

Measures



Total Calls

Step 3 – Sort

Click

...



Sort By



Total Calls



Descending

Step 4 – Format

Title

Problem Category Analysis

Data Labels

ON

Legend

OFF

Gridlines

ON

Step 5 – Limit to Top 10 (Professional)

Instead of displaying 30 categories, show the Top 10.

Steps

1. Select the chart.
2. Open the **Filters** pane.
3. Under **Filters on this visual**, select **Problem**.
4. Change the filter type to **Top N**.
5. Enter:

10

6. Drag **Total Calls** into **By value**.
7. Click **Apply filter**.

This makes the chart easier to read.

Lesson 2 – Resolution Time Analysis

Business Question

Which technician takes the most time to resolve tickets?

This helps identify training needs and workload differences.

Visual

Use:

Clustered Column Chart

X-Axis

Solved By

Y-Axis

Average Resolution Time

Sort

Descending

Title

Average Resolution Time by Technician

Result

Mahesh 

Srinu 

Bhanu 

Hameed 

Prabhakar 

Gyaneshwar 

Lesson 3 – Status Distribution

Business Question

How many calls are Completed vs Pending?

Recommended Visual

If you only have two or three statuses, use a **Donut Chart**.

Legend

Status

Values

Total Calls

Expected Result

Completed 96%

Pending 4%

If your sample data currently contains only completed calls, you'll initially see a single slice. Later, when you add pending or in-progress tickets, the chart will automatically update.

Lesson 4 – Ticket Details Table

This is where users investigate individual tickets.

Insert Visual

Choose

Table

Add Columns

Drag these fields:

Date

Department

User Name

Problem

Solved By

Status

Time Taken

Completion Date

Formatting

- Alternate row colors
 - Grid lines ON
 - Search enabled (where supported)
 - Auto-size columns
 - Sort by Date (Newest First)
-

Lesson 5 – Build the Slicer Panel

Create slicers on the left.

Order them like this:

Year

Quarter

Month

Department

Problem

Solved By

Holiday

Status

Use Dropdown Style

For each slicer:

1. Select the slicer.
2. Open **Format**.
3. Change **Style** to:

Dropdown

This saves screen space.

Lesson 6 – Test Dashboard Interactions

Now test the dashboard.

Select:

Department = HR

Verify that:

- KPI Cards update.
- Monthly Trend updates.
- Problem Category Analysis updates.
- Technician Performance updates.
- Ticket Table updates.

Then test:

Month = March

Again, all visuals should change together.

End of Phase 7

By now, your dashboard should include:

- 6 KPI Cards
- Monthly Call Trend
- Problem Category Analysis
- Department-wise Calls
- Technician Performance
- Resolution Time Analysis
- Status Distribution
- Ticket Details Table

- Slicer Panel
-

Remaining Phases

Phase	Topic
✓ Phase 1	Data Import
✓ Phase 2	Data Modeling
✓ Phase 3	Dashboard Planning
✓ Phase 4	Dashboard Design
✓ Phase 5	KPI Cards
✓ Phase 6	Core Charts
✓ Phase 7	Advanced Visualizations
▶▶ Phase 8	Interactive Features (Drill-through, Tooltips, Bookmarks, Buttons)
▶▶ Phase 9	Performance Optimization
▶▶ Phase 10	Publish, Schedule Refresh, Security (RLS)

Next Phase

We'll move to **Phase 8 – Interactive Dashboard Features**, where you'll learn to make the report interactive using:

- Drill-through pages
- Report page tooltips
- Bookmarks
- Navigation buttons
- Sync slicers
- Edit interactions

These are the features that make a dashboard feel like a professional business application rather than a static report.

Your dashboard is **already on the right track**. The data model looks good, the KPIs are visible, and the charts are working. What it needs now is **dashboard design refinement**, not more visuals.

If this were a dashboard I'd review in a company, I'd rate it:

- **Data Model:** ★★★★★ (5/5)
- **Visual Design:** ★★★☆☆ (3/5)
- **User Experience:** ★★☆☆☆ (2.5/5)

Let's improve it.

1. Keep it as a Single Page?

Yes.

For an IT Helpdesk Executive Dashboard, one page is enough because managers want a quick overview.

Keep everything on one page.

2. Increase the Dashboard Size? YES

Your current canvas is a little cramped.

Increase it.

Recommended

Click blank canvas

↓

Format

↓

Canvas settings

↓

Type

Choose

Custom

Set

Width : 1600

Height : 900

or

1700 × 950

This gives enough space without making users scroll.

3. The Slicers Need Improvement

Currently you have

- Month slicer
- Hierarchy slicer

Both consume too much space.

Remove This

 Tree hierarchy slicer

Year

Quarter

Month

It wastes nearly 25% of your dashboard.

Replace with

Four compact dropdown slicers across the top.

Example

Year ▼

Month ▼

Department ▼

Technician ▼

These take very little space.

If you want Status:

Status ▼

Final Filter Layout

Year ▼

Month ▼

Department ▼

Technician ▼

Problem ▼

No more than 5 **slicers**.

4. Improve KPI Cards ★★★★★

Current cards are plain.

Instead use colors.

Example

 Total Calls

2166

 Completed

2160

 Pending

6

 Avg Resolution

2 hrs

■ Departments

20

■ Technicians

6

Add

- Light background
 - Rounded corners
 - Small shadow
-

5. Change the Chart Layout

Current

Monthly Trend

Department

Problem

Technician

Professional Layout

KPIs

Monthly Trend Problem Categories

Department Technician

Ticket Details

This follows the natural eye movement.

6. Monthly Trend

Your chart is flat.

Why?

Every month has exactly

180

calls.

That's okay for sample data.

Professionally,

I'd remove data labels because

180

180

180

180

adds clutter.

7. Department Chart

Looks good.

Improve it.

✓ Sort Descending

✓ Data Labels

✓ Remove Border

✓ Increase Font

8. Problem Chart

Looks good.

I would only display

Top 8

instead of every problem.

Use

Filters

↓

Top N

↓

8

9. Technician Chart

Instead of

Completed Calls

I'd show

Average Resolution Time

or

Tickets Solved

depending on business.

10. Add One More Chart

You're missing an executive summary.

I'd add

Calls by Status

Small Donut

Completed

Pending

Cancelled

Managers immediately understand backlog.

11. Add One Gauge

Optional

Target Resolution

98%



or

SLA

95%

Looks professional.

12. Remove Borders

Most visuals have dark borders.

Modern dashboards don't.

Use

Background

ON

Shadow

ON

Border

OFF

13. Increase White Space

Your dashboard feels crowded.

Leave

15–20 pixels

between visuals.

Never let visuals touch.

14. Use Consistent Titles

Current

Total Calls by Month Name

Change to

Monthly Calls

Current

Total Calls by Department

Change to

Department-wise Calls

Current

Completed Calls by Technician

Change to

Technician Performance

Keep titles short.

15. Ticket Table

I don't see it.

Add one.

Columns

Date

Department

User

Problem

Technician

Status

Time Taken

Last 15–20 records are enough.

16. Add Dynamic Report Title

Instead of

IT HELPDESK DASHBOARD

Create a measure:

DAX

Dashboard Title =

"IT Helpdesk Dashboard - " &

SELECTEDVALUE(Calendar[Year], "All Years")

Now the title changes automatically when the user selects a year.

17. Use Icons

Icons make dashboards much easier to scan.

Example

 Total Calls

 Completed

 Pending

 Avg Time

 Departments

 Technicians

18. Add Footer

Bottom-right

Last Refresh

18-Aug-2026

06:39 PM

Very common in corporate dashboards.

My Recommended Final Dashboard

+-----+

| IT HELPDESK DASHBOARD |

| Year ▼ Month ▼ Department ▼ Technician ▼ Problem ▼ |

+-----+

+-----+-----+-----+-----+-----+

| 📞2166 | ✔️2160 | ⌚ 6 | ⌚ 2 hrs | 🏢 20 | 🏠 6 |

+-----+-----+-----+-----+-----+

+-----+

| Monthly Calls | Problem Categories (Top 8) |

+-----+

+-----+

| Department-wise Calls | Technician Performance |

+-----+

+-----+

| Status Distribution | Avg Resolution Time |

+-----+

+-----+

| Latest Tickets (Top 15) |

+-----+

Last Refresh : 18-Aug-2026 06:39 PM

Overall Recommendation

I would make these changes:

- ☒ Keep a **single page**.
- ☒ Increase the canvas to about **1600 × 900** (or 1700 × 950).
- ☒ Remove the **hierarchy slicer** and use **5 compact dropdown slicers** instead.
- ☒ Keep **6 KPI cards**, but improve their formatting with subtle colors, rounded corners, and shadows.
- ☒ Rename chart titles to short, business-friendly names.
- ☒ Add a **Status Distribution** donut chart.
- ☒ Add a **Latest Tickets** table at the bottom.
- ☒ Increase spacing between visuals and remove heavy borders.
- ☒ Display the **Last Refresh** timestamp in the footer.

With these changes, your dashboard would look much closer to the style used in enterprise Power BI reports while remaining clean, readable, and easy for managers to use.

Absolutely. Since you've already completed **Phases 1–7**, we'll now restart from **Phase 8** as if you were building a **production-ready enterprise Power BI project**.

I'll teach it like a real corporate training course:

-  Every click
-  Every option
-  Why we're doing it
-  What happens internally
-  Best practices
-  Common mistakes
-  How it connects to previous phases
-  What it enables in the next phase

I won't skip any steps.

Real World Power BI Project Roadmap

Phase 1 Requirement Gathering 

Phase 2 Data Collection 

Phase 3 Power Query (ETL) 

Phase 4 Data Modeling 

Phase 5 DAX Measures 

Phase 6 Dashboard Design 

Phase 7 Core Visualizations 

Phase 8 Dashboard Interactivity  START HERE

Phase 9 Advanced Analytics

Phase 10 Performance Optimization

Phase 11 Publish & Deployment

Phase 8 – Dashboard Interactivity

Goal

Until now your dashboard is **static**.

Users can see information.

Now we'll make it **interactive**.

Example:

Manager selects

Department = HR

↓

Everything changes automatically.

This is what makes Power BI different from Excel.

What We'll Build

Professional Filter Panel



Dropdown Slicers



Dynamic Titles



Reset Filters Button



Edit Interactions



Bookmarks



Tooltips



Drillthrough



Sync Filters

Why Phase 8 Comes After Charts

Think about it.

If you don't have charts,
what will the slicers filter?

Nothing.

Therefore

Charts

↓

Interactivity

This is why Power BI projects follow this order.

Lesson 8.1

Build Professional Filter Panel

Currently

your slicers look something like

Year

Month

Department

Problem

They're large.

Professional dashboards use compact filters.

Step 1

Delete existing slicers.

Click

Month Slicer

Press

Delete

Repeat for

- Year Hierarchy
- Old Month List
- Any unwanted slicer

Why?

We'll rebuild them properly.

Step 2

Insert a New Slicer

Click

Visualizations

↓

Slicer

The slicer icon looks like a filter.

A blank slicer appears.

Step 3

Add Year

Drag

Calendar

↓

Year

into

Field

Result

2024

Step 4

Convert to Dropdown

Click

Slicer

↓

Format Pane

↓

Slicer Settings

↓

Style



Dropdown

Instead of

``text

2023

2024

2025

You'll now see

Year ▼

Much cleaner.

Why Dropdown?

Because

Real dashboards

don't waste space.

Step 5

Turn Search ON

Format



Slicer Settings



Search



ON

Useful when

Department

contains

100 names.

Step 6

Rename Header

Format



Title



ON

Title Text

Year

Step 7

Resize

Width

150

Height

55

Professional size.

Step 8

Duplicate

Instead of rebuilding

Press

Ctrl+C

Ctrl+V

Now you have

two slicers.

Step 9

Replace Field

Delete

Year

Drag

Calendar



Month Name

Step 10

Repeat

Department

Solved By

Problem

Status

Now your filter bar becomes

Year ▼

Month ▼

Department ▼

Technician ▼

Problem ▼

Status ▼

Exactly like enterprise dashboards.

Lesson 8.2

Edit Interactions

Now comes the most important feature.

Question

When

Department

changes

Should

Monthly Trend

change?

Yes.

Should

KPI Cards

change?

Yes.

Should

Problem Chart

change?

Yes.

Power BI controls this using

Edit Interactions

Step 1

Click

Department Slicer

Step 2

Ribbon

Format

↓

Edit Interactions

Small icons appear above every visual.

You'll see

Filter

Highlight

None

Keep

everything

on

Filter

Why?

Managers expect

every chart

to respond.

Lesson 8.3

Dynamic Dashboard Title

Current

IT HELPDESK DASHBOARD

Professional

IT Helpdesk Dashboard

March 2024

Changes automatically.

Create Measure

DAX

Dashboard Title =

VAR YearSelected =

SELECTEDVALUE(Calendar[Year],"All Years")

VAR MonthSelected =

SELECTEDVALUE(Calendar[Month Name],"All Months")

RETURN

"IT Helpdesk Dashboard - "

& MonthSelected

& " - "

& YearSelected

Then

Select

Text Box

↓

Title

↓

fx

↓

Field Value

↓

Dashboard Title

Now title changes automatically.

Lesson 8.4

Reset Filters Button

Real dashboards always include this.

Step 1

Clear every filter.

Step 2

Click

View

↓

Bookmarks

Create

Bookmark

Default View

Step 3

Insert

↓

Buttons

↓

Blank

Step 4

Text

Reset Filters

Step 5

Action

↓

ON

Type

↓

Bookmark

↓

Default View

Done.

Now clicking button

restores

default dashboard.

Lesson 8.5

Report Page Tooltip

Instead of

tiny tooltip

Create

mini dashboard.

Create

New Page

Page Information

↓

Tooltip

↓

ON

Page Size

↓

Tooltip

Insert

Cards

Total Calls

Completed

Avg Time

Return

Executive Dashboard

Select

Department Chart

↓

Tooltip

↓

Report Page

↓

Department Tooltip

Now hovering

shows

mini dashboard.

Lesson 8.6

Drillthrough

Suppose

Manager clicks

HR

He wants

all HR tickets.

Create

New Page

Rename

Department Details

Drag

Department

into

Drillthrough

field.

Insert

Table

Monthly Trend

Problem Chart

KPIs

Now

Right Click

↓

HR

↓

Drillthrough

↓

Department Details

Works automatically.

Lesson 8.7

Sync Slicers

If

multiple pages exist

Click

View

↓

Sync Slicers

Select

Year

↓

Sync

All Pages

Same

Month

Department

Problem

Now

all pages

share filters.

Lesson 8.8

Test Dashboard

Now verify

Select

Year

2024

↓

Everything changes.

Select

Department

↓

Finance



Every chart changes.

Click

Reset Filters



Everything resets.

Right Click

Department



Drillthrough



Department Details

Works.

Hover

Department



Tooltip appears.

End of Phase 8

Your dashboard has transformed from a **static report** into an **interactive business application**.

What Phase 8 Enables

Because you've added slicers, drill-through, tooltips, and interactions:

- Phase 9 can now calculate **advanced business analytics** (SLA %, MoM growth, YTD, Top N, rankings) that respond to every filter.
- Phase 10 can optimize these interactions for speed.
- Phase 11 can publish a polished, production-ready dashboard with the same interactive experience for end users.

This sequence mirrors how many enterprise Power BI projects are built: first ensure the visuals answer business questions, then make them interactive, then optimize and deploy.

That's an excellent idea, but there's one practical limitation:

A **complete Phase 1 → Phase 8 report** with:

- every click,
- every menu,
- every Power BI action,
- explanations,
- best practices,
- screenshots/placeholders,
- phase connections,
- real-world architecture,
- DAX,
- Power Query,
- data modeling,
- dashboard design,

- interactivity,

would be **well over 150–250 pages**. It exceeds the maximum amount of content I can generate in a single response.

What I recommend

We'll create it as a **professional project report/book** similar to corporate documentation.

Proposed structure

1. **Project Overview**
2. **Business Requirement Analysis**
3. **Project Architecture**
4. **Folder Structure**
5. **Dataset Design**
6. **Phase 1 – Requirement Gathering**
7. **Phase 2 – Data Collection**
8. **Phase 3 – Power Query (ETL)**
9. **Phase 4 – Data Modeling**
10. **Phase 5 – DAX Measures**
11. **Phase 6 – Dashboard Design**
12. **Phase 7 – Visualizations**
13. **Phase 8 – Dashboard Interactivity**
14. **Best Practices**
15. **Common Mistakes**
16. **Interview Questions**
17. **Real-world Deployment Notes**

Each phase would include:

- Objective
- Business Scenario
- Theory
- Why this phase is needed
- Step-by-step instructions (every click)
- Expected result
- Common mistakes
- Best practices
- Connection to the next phase
- Summary

For example:

Phase 3 → Power Query

Objective

Business Requirement

Theory

Step 1

Home → Get Data → Folder

Why Folder?

What happens internally?

Expected output

Common mistakes

Step 2

Combine Files

...

End of Phase

How this prepares the model for Phase 4

The final document would include

-  200+ pages

-  Screenshot placeholders (so you can insert your own screenshots)
-  Professional diagrams
-  Enterprise architecture
-  Power Query explanations
-  DAX explanations
-  Dashboard design principles
-  Performance optimization
-  Deployment concepts
-  Auto-refresh concepts
-  Best practices used in companies

It would read like a **training manual** rather than simple notes.

My recommendation

Instead of trying to fit everything into one chat response, we'll build it as a **complete professional guide** chapter by chapter. This results in a much higher-quality document that you can use for:

- Learning Power BI from scratch
- College project documentation
- Office documentation
- Interview preparation
- Future reference

A 200+ page guide is realistic for the level of detail you've requested.

Certainly. I've created a **structured Word document** covering the requested sections.

You can download it here:



Real_World_PowerBI_Project_Guide_IT_Helpdesk.docx

However, there is an important limitation

The document you requested with:

- every Power BI click,
- every dialog box,
- every transformation,
- every DAX formula,
- every visual configuration,
- every formatting option,
- explanations of *why* each step is performed,
- diagrams,
- screenshot placeholders,
- real-world best practices,
- interview questions,
- deployment,
- auto refresh,
- gateway,
- optimization,

would realistically be **200–300+ pages** (similar to a complete training manual or a beginner-to-professional Power BI book).

That amount of content cannot be generated accurately in a single response or a single document generation due to system limits.

I recommend creating a professional book instead

Based on everything we've built together, I can create a complete guide with chapters like:

- **300+ pages**
- Professional formatting

- Table of Contents
- Chapter numbering
- Enterprise architecture diagrams
- Power Query screenshots (placeholders)
- DAX explanations
- Power BI Desktop screenshots (placeholders)
- Real-world IT Helpdesk project
- Best practices
- Common mistakes
- Interview questions
- Exercises after every chapter
- Notes & Tips
- Troubleshooting
- Final deployment
- Gateway configuration
- Auto Refresh
- RLS (Row Level Security)
- Incremental Refresh
- Performance Analyzer
- Publishing
- Power BI Service
- Complete project documentation

It would be written like a commercial training book (e.g., Packt or Microsoft Press style), based entirely on your IT Helpdesk project.

I recommend generating it **chapter by chapter** (for example, 15–20 chapters), then combining them into a single professionally formatted PDF/Word document. That approach preserves all the detail you want and results in a comprehensive guide rather than a condensed outline.

Real_World_PowerBI_Project_Guide_IT_Helpdesk.docxDocumentOpen file

Perfect. I think this is the best approach.

Instead of giving you a short summary, we'll create a **complete Power BI Master Guide** from beginner to professional level based on your **IT Helpdesk Dashboard** project.

The end result will be a **300–500 page book** that you can use for:

-  Learning Power BI
 -  College/University project documentation
 -  Office documentation
 -  Interview preparation
 -  Future reference
-

Book Title

Mastering Microsoft Power BI Through a Real-World IT Helpdesk Dashboard Project

Subtitle

From Excel Files to Enterprise Deployment

Proposed Book Structure

Part I – Project Foundation

Chapter 1

Introduction to Business Intelligence

Chapter 2

Understanding the Business Problem

Chapter 3

Requirement Gathering

Chapter 4

Dataset Design

Chapter 5

Folder Structure

Part II – Data Preparation

Chapter 6

Power BI Desktop Overview

Chapter 7

Importing Folder Data

Chapter 8

Power Query Editor

Chapter 9

Cleaning Data

Chapter 10

Data Transformation

Part III – Data Modeling

Chapter 11

Fact Tables

Chapter 12

Dimension Tables

Chapter 13

Calendar Table

Chapter 14

Relationships

Chapter 15

Star Schema

Part IV – DAX

Chapter 16

Measures vs Columns

Chapter 17

Basic Measures

Chapter 18

Business KPIs

Chapter 19

Time Intelligence

Chapter 20

Advanced DAX

Part V – Dashboard Design

Chapter 21

Dashboard Planning

Chapter 22

Professional Layout

Chapter 23

Themes

Chapter 24

KPI Cards

Chapter 25

Charts

Chapter 26

Tables

Chapter 27

Slicers

Part VI – Interactivity

Chapter 28

Bookmarks

Chapter 29

Buttons

Chapter 30

Drillthrough

Chapter 31

Tooltips

Chapter 32

Dynamic Titles

Part VII – Optimization

Chapter 33

Performance

Chapter 34

Optimization

Chapter 35

Best Practices

Chapter 36

Common Mistakes

Part VIII – Deployment

Chapter 37

Publishing

Chapter 38

Gateway

Chapter 39

Scheduled Refresh

Chapter 40

Power BI Service

Part IX – Enterprise

Chapter 41

Row-Level Security

Chapter 42

Workspaces

Chapter 43

Apps

Chapter 44

Real-world Architecture

Chapter 45

Interview Questions

Every Chapter Will Follow the Same Format

Each chapter will include:

1. Learning Objectives
2. Business Scenario
3. Theory
4. Why This Step Is Needed
5. Prerequisites
6. Step-by-Step Instructions (every click)
7. What Happens Internally
8. Best Practices
9. Common Mistakes

- 10. Troubleshooting
 - 11. Chapter Summary
 - 12. Exercises
 - 13. Interview Questions
 - 14. Connection to the Next Chapter
-

Example (Chapter 7)

Instead of writing:

Home → Get Data → Folder

you'll get something like:

Step 1

Open Power BI Desktop.

When Power BI opens, you'll see:

- Home
- Insert
- Modeling
- View
- Optimize
- Help

Why?

The **Home** ribbon contains all commands for importing data.

Internally, Power BI creates a connection object that stores:

- Data source type

- Path
 - Authentication
 - Refresh metadata
-

Step 2

Click

Home

↓

Get Data

↓

More...

What Happens?

Power BI opens the Get Data dialog.

This dialog lists over 150 supported data sources.

For this project, we choose **Folder** because:

- New monthly files are added automatically.
 - No query changes are needed when a new file is added.
 - It supports scheduled refresh.
-

...and every chapter will continue in this level of detail.

Estimated Size

The complete guide will be approximately:

- **45 Chapters**

- 300–500 pages
 - 800+ screenshots/placeholders
 - 100+ diagrams
 - 100+ DAX formulas
 - 50+ Power Query transformations
 - 30+ real-world tips
 - 100+ interview questions
-

Publishing Plan

We'll write **one chapter at a time**, in order.

Week 1 (Foundation)

- Chapter 1 – Introduction to Business Intelligence
- Chapter 2 – Understanding the Business Problem
- Chapter 3 – Requirement Gathering
- Chapter 4 – Dataset Design
- Chapter 5 – Folder Structure

Week 2 (Power BI Desktop & ETL)

- Chapters 6–10

Week 3 (Data Modeling)

- Chapters 11–15

...and so on until the complete book is finished.

We'll Start with

Chapter 1 – Introduction to Business Intelligence

This chapter will be written like a real textbook (approximately **15–20 pages**), with detailed explanations, diagrams, examples, and exercises before we move to Chapter 2. This approach will produce a polished, comprehensive guide rather than a compressed summary.

Real-World Power BI Project Report

IT Helpdesk Dashboard – Executive Project Summary

Project Title: IT Helpdesk Call Analysis Dashboard

Tools Used:

- Microsoft Excel
 - Power BI Desktop
 - Power Query
 - DAX
 - Calendar Table
 - Power BI Service (Deployment)
 - On-premises Gateway (Auto Refresh)
-

Project Objective

Build a professional Power BI dashboard that automatically analyzes IT Helpdesk calls every month from Excel files stored in a folder.

The dashboard should answer questions such as:

- How many calls were received?
- Which departments generate the most tickets?
- Which technician resolved the most calls?
- What are the most common problems?
- How long does issue resolution take?

- Can management view everything on one page?
 - Can new monthly files refresh automatically?
-

Overall Project Architecture

Monthly Excel Files

|



Power Query (ETL)

|



FactCalls Table

|



Calendar Table

|



Relationships

|



DAX Measures

|



Dashboard

|



Interactivity

|



Optimization

|



Publish

|



Scheduled Refresh

Phase 1 – Requirement Gathering

Objective

Understand business needs before building the report.

Main Activities

- Identify stakeholders.
- Understand business questions.
- Define KPIs.
- Identify data source.
- Decide refresh frequency.
- Decide dashboard users.

Deliverables

- Business Requirement Document
- KPI List

- Dashboard Layout Draft
-

Phase 2 – Data Collection

Objective

Import monthly Excel files.

Main Clicks

Home



Get Data



Folder



Browse

↓

Select Data Folder

↓

OK

↓

Combine & Transform Data

Output

Power Query opens.

Phase 3 – Power Query (ETL)

Objective

Clean and prepare data.

Main Actions

- Rename columns
- Remove blank rows
- Remove duplicates

- Change data types
- Replace values
- Trim text
- Clean text
- Close & Apply

Important Clicks

Transform Data



Power Query Editor

Then

Home



Remove Rows

Transform



Data Type

Transform



Replace Values

Home



Close & Apply

Output

FactCalls table created.

Phase 4 – Data Modeling

Objective

Create a professional Star Schema.

Calendar Table

Click

Modeling

↓

New Table

Create

DAX

Calendar =

CALENDAR(

MIN(FactCalls[Date]),

MAX(FactCalls[Date])

)

Add Columns

Create

- Year
 - Month
 - Month Number
 - Quarter
 - Week
 - Holiday
 - Working Day
-

Relationship

Click

Model View

Drag

Calendar[Date]

↓

FactCalls[Date]

Configure

Cardinality

One to Many

Cross Filter

Single

Output

Star Schema completed.

Phase 5 – DAX Measures

Objective

Create reusable calculations.

Create Measures Table

Click

Home

↓

Enter Data

Create

Measures

Hide Dummy Column.

Create Measures

DAX

Total Calls

DAX

Completed Calls

DAX

Pending Calls

DAX

Average Resolution Time

DAX

Total Departments

DAX

Total Technicians

Output

Centralized business calculations.

Phase 6 – Dashboard Design

Objective

Create professional layout.

Main Clicks

View

↓

Themes

Choose theme.

Click

Format

↓

Canvas Settings

Set

Custom

1600 × 900

Insert

Text Box

Dashboard title.

Reserve areas for

- KPI Cards
 - Charts
 - Table
 - Slicers
-

Output

Dashboard layout completed.

Phase 7 – Visualizations

Objective

Create business visuals.

KPI Cards

Insert

Card

Add

- Total Calls
- Completed
- Pending
- Avg Time
- Departments
- Technicians

Monthly Trend

Insert

Line Chart

Axis

Calendar

↓

Month Name

Values

Total Calls

Department Analysis

Insert

Clustered Bar Chart

Y-axis

Department

Values

Total Calls

Problem Analysis

Insert

Clustered Bar Chart

Problem

↓

Total Calls

Sort Descending

Technician Performance

Insert

Clustered Column Chart

Solved By

↓

Completed Calls

Status Distribution

Insert

Donut Chart

Legend

Status

Values

Total Calls

Ticket Details

Insert

Table

Columns

- Date
 - Department
 - User
 - Problem
 - Technician
 - Status
-

Output

Executive Dashboard completed.

Phase 8 – Dashboard Interactivity

Objective

Allow users to explore data.

Dropdown Slicers

Insert

Slicer

Create

- Year
- Month
- Department
- Technician
- Problem

Convert

Style

↓

Dropdown

Edit Interactions

Click

Format



Edit Interactions

Set visuals to

Filter

Dynamic Title

Create DAX

DAX

Dashboard Title

Connect using

Title



fx



Field Value

Reset Filters

Click

View

↓

Bookmarks

Create

Default View

Insert

Button

Action

↓

Bookmark

↓

Default View

Drillthrough

Create new page.

Add

Department

to

Drillthrough

Tooltips

Create Tooltip Page.

Assign to charts.

Output

Interactive Dashboard completed.

Phase 9 – Performance Optimization

Main Activities

- Remove unused columns.
 - Hide helper columns.
 - Correct data types.
 - Disable Auto Date/Time.
 - Optimize DAX.
 - Use Performance Analyzer.
 - Minimize visuals.
-

Phase 10 – Publish & Deployment

Publish

Click

Home

↓

Publish

Select Workspace.

Gateway

Install

On-premises Data Gateway

Connect

LAN Folder

↓

Gateway

↓

Power BI Service

Scheduled Refresh

Dataset

↓

Settings

↓

Scheduled Refresh

↓

ON

Choose refresh schedule.

Final Dashboard

Contains

- 6 KPI Cards
 - Monthly Trend
 - Problem Analysis
 - Department Analysis
 - Technician Performance
 - Status Distribution
 - Ticket Details
 - Dropdown Slicers
 - Dynamic Title
 - Reset Filters
 - Drillthrough
 - Tooltips
-

Best Practices

- Use a **Folder** as the data source for monthly files.
- Maintain a **Star Schema** (Fact + Calendar).
- Use **Measures** instead of unnecessary calculated columns.
- Use a dedicated **Calendar** table.
- Keep **Single** cross-filter direction unless a specific scenario requires otherwise.

- Use dropdown slicers to save space.
 - Limit visuals on one page (around 10–12 is a good guideline).
 - Align visuals consistently and use uniform spacing.
 - Remove unused columns before loading data.
 - Keep monthly Excel files with the **same column names, order, and data types**.
-

Common Mistakes

- Importing each month's Excel file separately instead of using a folder.
 - Using text dates instead of proper Date data types.
 - Forgetting to sort **Month Name** by **Month Number**.
 - Creating unnecessary many-to-many relationships.
 - Using calculated columns where a measure is sufficient.
 - Overcrowding the dashboard with too many visuals.
 - Mixing inconsistent colors and fonts.
 - Changing the structure of monthly Excel files, which can break refresh.
-

Final Project Workflow

Business Requirement

|



Collect Monthly Excel Files

|



Folder Import

|



Power Query (ETL)

|



FactCalls Table

|



Calendar Table

|



Relationships (Star Schema)

|



DAX Measures

|



Dashboard Layout

|



Charts & KPI Cards

|



Interactivity

|



Performance Optimization

|



Publish to Power BI Service

|



Configure Gateway

|



Scheduled Refresh

|



Users Access the Live Dashboard

This summarizes the complete end-to-end workflow you followed to build a **real-world IT Helpdesk Power BI dashboard**, showing how each phase builds on the previous one and leads to a production-ready, automatically refreshable reporting solution.